

Phenix Model Tools and Utilities

12 documentation pages

Generated from local Phenix documentation

Contents

1. Sequence assignment and linkage of neighboring segments with `assign_sequence`
2. Comparison of CA positions in two models allowing any order of fragments
3. `<string>`
4. Morphing a model with `morph_model`
5. `phenix.pdbtools`: PDB model manipulations and statistics
6. Processing AlphaFold2, RoseTTAFold and other predicted models
7. Model searching and morphing using secondary structure matching (SSM)
8. Model superposition and morphing using secondary structure matching (SSM) (deprecated, use `superpose_models`)
9. `phenix.superpose_maps`
10. Model superposition with optional morphing and trimming
11. Superimposing two PDB files with `superpose_pdb`s (deprecated, use `superpose_models`)
12. Trimming overlapping parts of a model

Sequence assignment and linkage of neighboring segments with `assign_sequence`

`assign_sequence.html`

Author(s)

- `assign_sequence`: Tom Terwilliger

Purpose

You can now carry out an improved sequence assignment of a model that you have already built with `phenix.assign_sequence`. Further, once the sequence has been assigned, this method will use the sequence and proximity to identify chains that should be connected, and it will connect those that have the appropriate relationships using the new loop libraries available in `phenix.fit_loops`. The result is that you may be able to obtain a more complete model with more chains assigned to sequence than previously.

`assign_sequence` is a command line tool for reanalyzing resolve sequence assignment for a model and a map including the non-crystallographic symmetry, exclusion of sequence by previously-assigned regions, and requirement for plausible distances and geometries between ends of fragments with assigned sequences. Additionally `assign_sequence` will use the `fit_loops` loop library to connect segments that are separated by a short loop.

Note: `assign_sequence` is designed to be used after resolve model-building in which residues that are not assigned to sequence are given residue numbers higher than any residue in the input sequence file. If you input a model not built by resolve or in phenix, or if you would like to completely redo the sequence assignment for your model, be sure to set "`allow_fixed_segments=False`".

NOTE: `assign_sequence` is normally called from `phenix.phase_and_build` but you can run it interactively if you want.

Usage

How `assign_sequence` works:

The starting point for `assign_sequence` is a set of segments of structure read in from the input model. `assign_sequence` then uses resolve to calculate the compatibility of each possible side chain with each residue in each segment. Then `assign_sequence` tests out possible combinations of alignments of all the segments in the input model and chooses the set of alignments that is most compatible with the density map, the number of NCS copies, and with the geometries and distances between ends of the segments.

Sequence probabilities:

`assign_sequence` uses the side-chain to map compatibility matrix calculated by resolve to assess the relative probabilities of each possible side chain at each position in the input model. Segments that are positively assigned to a sequence by resolve are (by default) maintained and used as anchors for further sequence assignment. All other segments have a relative probability associated with each possible alignment of the segment to the input sequence. The score for each alignment is the logarithm of this probability (essentially a log-likelihood LL score).

Connection scores:

Any pair of segments with some assignment of sequence to each segment has an additional score corresponding to the plausibility of a connection of the expected length existing between the segments. If the distance between ends is greater than can be bridged by the number of residues separating them, then the connection is not possible. If the connection is possible, it is scored based on the best density fit (CC) of a loop from the fit_loops loop library. This additional score is normally $10 \times \text{CC}$.

Generating sequence alignments and connectivities

assign_sequence starts with the segments with the most convincing assignments of sequence. Often these are those with sequence positively assigned by resolve; otherwise they are those with the highest-probability assignments. This yields a starting arrangement (sequence assignment for a set of segments). Then each possible sequence assignment of each unassigned segment is tested for compatibility with the existing arrangement and the one that is most compatible (based on the connections that would result, duplication of sequence, and sequence-map matching) is added to the arrangement. Optionally many arrangements can be built up in parallel, but often a very good one can be found simply by taking the top one at each step. This process is repeated until no additional segments can be added to the arrangement to yield an increase in log-likelihood score of (by default) 2 or greater.

NCS copies:

assign_sequence builds up a set of possible sequence assignments and connectivities that depends on the expected number of copies in the asymmetric unit of the crystal. If there is only one copy of the molecule in the crystal, then no residues in the sequence can be used more than once in sequence assignment. If there are N copies, then a residue can be used up to N times. If there are multiple copies, then each molecule must be self-consistent, with plausible distances and geometries relating each segment to the next.

Connecting segments:

Once a final arrangement is found, including NCS if applicable, all segments that are separated by short loops (typically 0-3 residues) are connected using loops from the fit_loops loop library. This yields longer segments of structure with sequences fully assigned. The resulting model then has side chains added to match the newly-assigned sequence and is written out.

Output files from assign_sequence

assign_sequence.pdb: A PDB file with your input model assigned to sequence (to the extent possible). Residues not assigned to sequence will be given a chain ID higher than those assigned, and they will be given residue numbers higher than any residue number in the sequence file.

Examples

Standard run of assign_sequence:

Running assign_sequence is easy. From the command-line you can type:

```
phenix.assign_sequence map_coeffs.mtz coords.pdb sequence.dat
```

If you want (or need) to specify the column names from your mtz file, you will need to tell assign_sequence what FP and PHIB (and optionally FOM) are, in this format:

```
phenix.assign_sequence map_coeffs.mtz coords.pdb \  
  labin="FP=2F0FCWT PHIB=PH2F0FCWT" sequence.dat
```

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

input_filesseq_file = None File with 1-letter code sequence of molecule. Chains separated by blank line or greater-than signpdb_in = None Optional starting PDB file (ends will be extended if present)mtz_in = None MTZ file with coefficients for a map map_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM)labin = "" For backward compatibility only. Use instead map_coeff_labels. Labin line for MTZ file with map coefficients. This is optional if assign_sequence can guess the correct coefficients for FP PHIB . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP. NOTE: myFP and myPHI must be adjacent in the mtz file.map_in = None CCP4 or MRC-style map fileprob_file = None File with sequence probability information from resolveec_hist_file = /Users/terwill/unix/PHENIX/build_38/modules/solve_resolve/ext_ref_files/ec_hist.pkl File with EC histogramsec_dict_file = None Input file with pre-calculated EC scoresss_pred_file = None File with secondary structure probability information in the format of raptorX (<http://raptorx.uchicago.edu/StructurePropertyPred/predict/>)linkage_file = None File with linkage information from combine_models Must have been run with the same input pdb file and the same value of min_segment_lengthloop_dict_file = None File with loop information from previous run as pickle filepair_object_dict_file = None pair object dict as pickle filechecked_connections_file = None checked connections dict as pickle filedensity_removed_mtz_in = None MTZ file with density_removed coefficients for a map comparison_model = None Comparison model (normally just for testing)ec_file = None EC file (format is residue i residue j EC value)output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.pdb_out = assign_sequence.pdb Output PDB filelog = None Output log fileparams_out = assign_sequence_params.eff Parameters file to rerun assign_sequenceoutput_loop_dict_file = None loop dict as pickle fileoutput_pair_object_dict_file = None pair object dict as pickle fileoutput_checked_connections_file = None checked connections dict as pickle fileoutput_ec_dict_file = None ec dict as pickle fileassignmentmap_all_connections = False Map all connecting parts, even those that are not linkedec_scale = None Scale on EC scores (compared to density analysis). If None, adjust to match SD of EC scores to SD of scores from density.fraction_ec_inter_chain = 0.1 Fraction of inter-chain EC values to save ncs_resolution = None Resolution for NCS identificationfind_ncs = False Try to find NCS in chains after sequence assignmentrange_to_keep = 4.0 Keep solutions with score within range_to_keep of the maximummax_keep = 10 Number of possibilities to keep in optimizationmax_write = 1 Number of possibilities to write out at endl linkage_score = 10. Score for creating a link between segmentsmax_linkage_score = 12. Maximum linkage score attainableloop_score = 10. Score for a loop is loop_score*(1.+loop_cc)max_overlap_rmsd = 2. Maximum rmsd for 3 residues on each end of loop-lib fitallowed_overlap = 4 Allowed overlap of segments allowed. At the end if there are overlaps they will be trimmed off.length_mismatch_penalty = 0 Decrease in linkage score if linkage is not correct lengthdepth_to_keep = 8.0 In full optimization solutions will be kept with score depth_to_keep + max_linkage_score below the bestmaximum_length_mismatch = 3 Maximum length mismatch in

linkagesmin_confidence = 0.9999 Sets required confidence level in a plac...

- input_fillesseq_file = None File with 1-letter code sequence of molecule. Chains separated by blank line or greater-than sign
 - pdb_in = None Optional starting PDB file (ends will be extended if present)
 - mtz_in = None MTZ file with coefficients for a map
 - map_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM)
 - labin = "" For backward compatibility only. Use instead map_coeff_labels. Labin line for MTZ file with map coefficients. This is optional if assign_sequence can guess the correct coefficients for FP PHIB . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP. NOTE: myFP and myPHI must be adjacent in the mtz file.
 - map_in = None CCP4 or MRC-style map file
 - prob_file = None File with sequence probability information from resolve
 - ec_hist_file = /Users/terwill/unix/PHENIX/build_38/modules/solve_resolve/ext_ref_files/ec_hist.pkl File with EC histograms
 - ec_dict_file = None Input file with pre-calculated EC scores
 - ss_pred_file = None File with secondary structure probability information in the format of raptorX (<http://raptorx.uchicago.edu/StructurePropertyPred/predict/>)
- map_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM)
- labin = "" For backward compatibility only. Use instead map_coeff_labels. Labin line for MTZ file with map coefficients. This is optional if assign_sequence can guess the correct coefficients for FP PHIB . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP. NOTE: myFP and myPHI must be adjacent in the mtz file.
- map_in = None CCP4 or MRC-style map file
- prob_file = None File with sequence probability information from resolve
- ec_hist_file = /Users/terwill/unix/PHENIX/build_38/modules/solve_resolve/ext_ref_files/ec_hist.pkl File with EC histograms
- ec_dict_file = None Input file with pre-calculated EC scores
- ss_pred_file = None File with secondary structure probability information in the format of raptorX (<http://raptorx.uchicago.edu/StructurePropertyPred/predict/>)

- linkage_file = None File with linkage information from combine_models Must have been run with the same input pdb file and the same value of min_segment_length
- loop_dict_file = None File with loop information from previous run as pickle file
- pair_object_dict_file = None pair object dict as pickle file
- checked_connections_file = None checked connections dict as pickle file
- density_removed_mtz_in = None MTZ file with density_removed coefficients for a map
- comparison_model = None Comparison model (normally just for testing)
- ec_file = None EC file (format is residue i residue j EC value)
- output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.pdb_out = assign_sequence.pdb Output PDB filelog = None Output log fileparams_out = assign_sequence_params.eff Parameters file to rerun assign_sequenceoutput_loop_dict_file = None loop dict as pickle fileoutput_pair_object_dict_file = None pair object dict as pickle fileoutput_checked_connections_file = None checked connections dict as pickle fileoutput_ec_dict_file = None ec dict as pickle file
- target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- pdb_out = assign_sequence.pdb Output PDB file
- log = None Output log file
- params_out = assign_sequence_params.eff Parameters file to rerun assign_sequence
- output_loop_dict_file = None loop dict as pickle file
- output_pair_object_dict_file = None pair object dict as pickle file
- output_checked_connections_file = None checked connections dict as pickle file
- output_ec_dict_file = None ec dict as pickle file
- assignmentmap_all_connections = False Map all connecting parts, even those that are not linkedec_scale = None Scale on EC scores (compared to density analysis). If None, adjust to match SD of EC scores to SD of scores from density.fraction_ec_inter_chain = 0.1 Fraction of inter-chain EC values to save ncs_resolution = None Resolution for NCS identificationfind_ncs = False Try to find NCS in chains after sequence assignmentrange_to_keep = 4.0 Keep solutions with score within range_to_keep of the maximummax_keep = 10 Number of possibilities to keep in optimizationmax_write = 1 Number of possibilities to write out at endlinkage_score = 10. Score for creating a link between segmentsmax_linkage_score = 12. Maximum linkage score attainableloop_score = 10. Score for a loop is loop_score*(1.+loop_cc)max_overlap_rmsd = 2. Maximum rmsd for 3 residues on each end of loop-lib fitallowed_overlap = 4 Allowed overlap of segments allowed. At the end if there are overlaps they will be trimmed off.length_mismatch_penalty = 0 Decrease in linkage score if linkage is not correct lengthdepth_to_keep = 8.0 In full optimization solutions will be kept with score depth_to_keep + max_linkage_score below the bestmaximum_length_mismatch = 3 Maximum length mismatch in linkagesmin_confidence = 0.9999 Sets required confidence level in a placement of a segment to keep the best one. convincing_score = 2. Score gain required to keep a sequence assignmentstarting_convincing_score = 10. Score gain required to keep an initial sequence assignmentuse_ec = None Use EC values in assigning sequence. Requires ec_file.use_density = True Use density in assigning sequence. Requires map coefficientsminimum_inter_residue_difference = None

Ignore EC values for residue differences shorter than this `maximum_inter_residue_difference = None`
 Ignore EC values for residue differences longer than this `subtract_expected_value = True` Subtract ec values for distances associated with intra-residue differences. Alternative is subtract ec values for all distances corresponding to residue differences. `match_ec_to_pdb = None` Align ec to pdb sequence and adjust ec residue numbers. Alternatives are no matching or `match_c_to_seq` `match_ec_to_seq = None` Align ec to input sequence and adjust ec residue numbers. Alternatives are no matching or `match_c_to_pdb`. NOTE: sequence file must have only one chain and start at residue 1 unless `start_res` is specified. `min_ec_length = 5` Minimum length of match between PDB and EC file `random_score = 20` Number of random distances to average for random scoring `max_separation_for_bins = 20` Maximum distance (in residues) between residues to consider in binning EC values `max_ec_seq_mismatch = 2` Maximum mismatches of sequence in EC database compared to PDB file or sequence file. `use_cb_for_ec = True` use C-beta positions for EC calculations `minimum_ec = -1000` Minimum EC to consider `minimum_length = 4` Minimum length of a segment to place `min_segment_length = 5` Segments shorter than this will be ignored on read-in. `max_levels = None` Number of segments to consider in building a complete sequence assignment (quick default = 6, otherwise 20) `max_indiv_tries_per_level = None` Number of sequence assignments to consider for each segment (quick default = 1, otherwise 3) `max_total_tries_per_level = None` Number of sequence arrangements to consider for all additional segments (quick default = 1, otherwise 6) `max_placements = 100` Number of placements of any segment to consider `max_final_placements = 20` Number of final arrangements to consider `check_ncs_with_offset = True` Check to verify that segments that seem to show NCS are actually different if offset by 1 residue. If protein is just helices then you might need to try `check=False` `list_only_complete = False` Only include complete arrangements; ignore those that have arrangements of some segments that are subsequently removed as incompatible `allow_fixed_segments = True` If True, then input segments with sequences assigned are kept fixed. If fix...

- `map_all_connections = False` Map all connecting parts, even those that are not linked
- `ec_scale = None` Scale on EC scores (compared to density analysis). If None, adjust to match SD of EC scores to SD of scores from density.
- `fraction_ec_inter_chain = 0.1` Fraction of inter-chain EC values to save
- `ncs_resolution = None` Resolution for NCS identification
- `find_ncs = False` Try to find NCS in chains after sequence assignment
- `range_to_keep = 4.0` Keep solutions with score within range_to_keep of the maximum
- `max_keep = 10` Number of possibilities to keep in optimization
- `max_write = 1` Number of possibilities to write out at end
- `linkage_score = 10`. Score for creating a link between segments
- `max_linkage_score = 12`. Maximum linkage score attainable
- `loop_score = 10`. Score for a loop is $\text{loop_score} * (1 + \text{loop_cc})$
- `max_overlap_rmsd = 2`. Maximum rmsd for 3 residues on each end of loop-lib fit
- `allowed_overlap = 4` Allowed overlap of segments allowed. At the end if there are overlaps they will be trimmed off.
- `length_mismatch_penalty = 0` Decrease in linkage score if linkage is not correct length
- `depth_to_keep = 8.0` In full optimization solutions will be kept with score `depth_to_keep + max_linkage_score` below the best
- `maximum_length_mismatch = 3` Maximum length mismatch in linkages

- `min_confidence = 0.9999` Sets required confidence level in a placement of a segment to keep the best one.
- `convincing_score = 2`. Score gain required to keep a sequence assignment
- `starting_convincing_score = 10`. Score gain required to keep an initial sequence assignment
- `use_ec = None` Use EC values in assigning sequence. Requires `ec_file`.
- `use_density = True` Use density in assigning sequence. Requires map coefficients
- `minimum_inter_residue_difference = None` Ignore EC values for residue differences shorter than this
- `maximum_inter_residue_difference = None` Ignore EC values for residue differences longer than this
- `subtract_expected_value = True` Subtract ec values for distances associated with intra-residue differences. Alternative is subtract ec values for all distances corresponding to residue differences.
- `match_ec_to_pdb = None` Align ec to pdb sequence and adjust ec residue numbers. Alternatives are no matching or `match_c_to_seq`
- `match_ec_to_seq = None` Align ec to input sequence and adjust ec residue numbers. Alternatives are no matching or `match_c_to_pdb`. NOTE: sequence file must have only one chain and start at residue 1 unless `start_res` is specified.
- `min_ec_length = 5` Minimum length of match between PDB and EC file
- `n_random_score = 20` Number of random distances to average for random scoring
- `max_separation_for_bins = 20` Maximum distance (in residues) between residues to consider in binning EC values
- `max_ec_seq_mismatch = 2` Maximum mismatches of sequence in EC database compared to PDB file or sequence file.
- `use_cb_for_ec = True` use C-beta positions for EC calculations
- `minimum_ec = -1000` Minimum EC to consider
- `minimum_length = 4` Minimum length of a segment to place
- `min_segment_length = 5` Segments shorter than this will be ignored on read-in.
- `max_levels = None` Number of segments to consider in building a complete sequence assignment (quick default = 6, otherwise 20)
- `max_indiv_tries_per_level = None` Number of sequence assignments to consider for each segment (quick default = 1, otherwise 3)
- `max_total_tries_per_level = None` Number of sequence arrangements to consider for all additional segments (quick default = 1, otherwise 6)
- `max_placements = 100` Number of placements of any segment to consider
- `max_final_placements = 20` Number of final arrangements to consider
- `check_ncs_with_offset = True` Check to verify that segments that seem to show NCS are actually different if offset by 1 residue. If protein is just helices then you might need to try `check=False`
- `list_only_complete = False` Only include complete arrangements; ignore those that have arrangements of some segments that are subsequently removed as incompatible
- `allow_fixed_segments = True` If True, then input segments with sequences assigned are kept fixed. If `fix_known=False`, then assigned segments are identified by sequence numbers less than or equal to the longest segment in the sequence file.
- `all_are_fixed = False` Fix all segments. Note: will fail if multiple chains

- `fix_known = False` If True, then all input segments with except those marked as `segid=UNK` will be considered fixed. (Instead of identifying them based on sequence number in input file). Forces `allow_fixed_segments` to be True.
- `keep_connectivity = False` This is very useful if you expect your model to have the same connectivity as your template. If True, then input segments are kept in the order found in the input PDB and kept assigned to the original chains, but their assignments may change otherwise (allowing insertions/deletions). NOTE: If True, then `allow_fixed_segments=False` unless `fix_known=True`
- `include_chain_u_in_keep_connectivity = False` If True, chains with chainID of 'U' are included in analysis with `keep_connectivity=True`
- `optimize_arrangements = None` Try to optimize arrangements at the end, including removal of uncertain segments
- `use_connectivity_in_optimize = True` If True and `keep_connectivity` is True and `optimize_arrangements` is True , then connectivity will be used in chain optimization
- `remove_uncertain_segments = True` If True, then remove uncertain segments. NOTE: at very end of iteration, if any, then the removal of uncertain segments is determined by `iteration.remove_uncertain_at_end`.
- `optimize_sequence_alignment = False` If True, then try to align fragments to template sequence Only appropriate if fully assigned model is input (not likely)
- `minimize_alignment_changes = False` If True, then try to keep sequence numbers matched to original as much as possible.
- `allow_longer_connections = True` If a connection segment is available, try with lower scoring also `n+1,n+2,n+3` connections. This may be a good idea because loops are often built short by one or a few residues.
- `replace_side_chains = True` At the end of sequence assignment identify side-chain rotamers and replace existing side-chains
- `replace_direct_joins = False` Use fit-loops to rebuild all junctions that are joined flush
- `short_segment_length = 12` Definition of a short segment
- `max_loop_length = 8` Maximum length of loop to try to fit
- `n_random_loop = 500` Number of loop versions to build
- `max_unassigned_short_segments = 20` Maximum number of segments `short_segment_length` or fewer residues that are not assigned to sequence to consider in connections. Keeping too many can make the analysis take a very long time.
- `compare_only = False` Just compare input model to comparison model
- `reset_sequence = False` Adjust sequence numbering and sequence of segments in `pdb_in` based on sequence numbers of matching positions in `comparison_model`. Cannot be used with `compare_only`.
- `make_unique = None` Make segments of input model unique (no overlapping) residues. Normally keep this at None. Used in iteration of `assign_sequence`.
- `start_res = None` Starting residue number of sequence file. Used only for `compare_only` with `ec` weighting
- `ignore_ter_or_break = None` Ignore TER/BREAK records in PDB file. Use only residue numbers and chain ID to identify breaks in `seq_prob` steps

- `iteration`
`iterative_assignment = False` You can iteratively assign sequence and fit loops This can improve the assignment, but will take longer
`cycles = 3` Cycles of iteration
`start_step = *assign_sequence` `fit_loops` `insert_loops` `get_connections` You can decide where to start in the cycle
`end_step = *None` `assign_sequence` `fit_loops` `insert_loops` `get_connections` You can specify last step to carry out in iteration
`skip_step = None` You can specify steps to skip in iteration
`assign_sequence_file = None` File partially assigned to sequence (output of `assign_sequence`)
`loops_file = None` File with loops to insert
`assign_sequence_insertion_file = None` File partially assigned to sequence with insertions
`connections_file_list = None` list of files with possible connections to consider
`only_assign_sequence_on_last_cycle = True` Just run `assign_sequence` step on last iteration cycle
`build_outside_model = True` Build outside model in `get_connections`
`build_outside_model_once = True` Run build outside model a maximum of one time (Run once if `build_outside_model=True`, never if `build_outside_model=False`)
`remove_uncertain_at_end = False` If True, then remove uncertain segments at very end of iteration (in addition to removing them along the way if `remove_uncertain_segments=True`)
`connect_all_segments = True` Connect all segments sequentially in `get_connections`
`fill_in_gaps = True` After optimize and keep connectivity, try to fill in gaps
`standard_gap_score = 0`. Score for filling default gap
`decrease_tries_with_levels = False` Reduce number of tries each level of search
- `iterative_assignment = False` You can iteratively assign sequence and fit loops This can improve the assignment, but will take longer
- `cycles = 3` Cycles of iteration
- `start_step = *assign_sequence` `fit_loops` `insert_loops` `get_connections` You can decide where to start in the cycle
- `end_step = *None` `assign_sequence` `fit_loops` `insert_loops` `get_connections` You can specify last step to carry out in iteration
- `skip_step = None` You can specify steps to skip in iteration
- `assign_sequence_file = None` File partially assigned to sequence (output of `assign_sequence`)
- `loops_file = None` File with loops to insert
- `assign_sequence_insertion_file = None` File partially assigned to sequence with insertions
- `connections_file_list = None` list of files with possible connections to consider
- `only_assign_sequence_on_last_cycle = True` Just run `assign_sequence` step on last iteration cycle
- `build_outside_model = True` Build outside model in `get_connections`
- `build_outside_model_once = True` Run build outside model a maximum of one time (Run once if `build_outside_model=True`, never if `build_outside_model=False`)
- `remove_uncertain_at_end = False` If True, then remove uncertain segments at very end of iteration (in addition to removing them along the way if `remove_uncertain_segments=True`)
- `connect_all_segments = True` Connect all segments sequentially in `get_connections`
- `fill_in_gaps = True` After optimize and keep connectivity, try to fill in gaps
- `standard_gap_score = 0`. Score for filling default gap
- `decrease_tries_with_levels = False` Reduce number of tries each level of search
- `directory`
`temp_dir = "temp_dir"` Optional temporary work directory
`output_dir = ""` Optional output directory
- `temp_dir = "temp_dir"` Optional temporary work directory
- `output_dir = ""` Optional output directory

- crystal_inforesolution = 0. high-resolution limit for map calculationsolvent_fraction = 0.5 solvent fractionchain_type = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms)ncs_copies = 1 NCS copies
- resolution = 0. high-resolution limit for map calculation
- solvent_fraction = 0.5 solvent fraction
- chain_type = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms)
- ncs_copies = 1 NCS copies
- model_buildingdist_max = 20. Maximum distance ends can be apart to consider for linkingmax_loop_lib_gap = 3 Maximum number of residues in working loop library (This must match loop libraries that are available)
- dist_max = 20. Maximum distance ends can be apart to consider for linking
- max_loop_lib_gap = 3 Maximum number of residues in working loop library (This must match loop libraries that are available)
- controlverbose = False Verbose outputquick = True Run quicklyraise_sorry = False Raise sorry if problemsdebug = False Debugging outputdry_run = False Just read in and check parameter namesnproc = 1 You can specify the number of processors to use check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to endwait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....resolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "; background = None run jobs in background or not (if nproc is greater than 1) Usually set automatically. If run_command is sh or csh, Truerun_command = "sh " Command for running jobs (e.g., sh or qsub)resolve_size = 24 Size of resolve to use. em_side_density = False Use EM side chain density
- verbose = False Verbose output
- quick = True Run quickly
- raise_sorry = False Raise sorry if problems
- debug = False Debugging output
- dry_run = False Just read in and check parameter names
- nproc = 1 You can specify the number of processors to use
- check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end
- wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....
- resolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' ";
- background = None run jobs in background or not (if nproc is greater than 1) Usually set automatically. If run_command is sh or csh, True

- `run_command = "sh "` Command for running jobs (e.g., `sh` or `qsub`)
- `resolve_size = 24` Size of resolve to use.
- `em_side_density = False` Use EM side chain density
- `non_user_paramsprint_citations = True` Print citation information at end of run
- `print_citations = True` Print citation information at end of run

Comparison of CA positions in two models allowing any order of fragments

chain_comparison.html

Author(s)

- chain_comparison: Tom Terwilliger

Purpose

Identify how many of the CA atoms in one model match CA atoms in another model. Separately identify how many of these go in the same direction and how many of the residue names match.

Usage

How chain_comparison works:

chain_comparison runs normally selects the unique part of the target model (the first model specified), then it takes the entire query model and counts how many residues in the target model are matched by a residue in the query model. If NCS is supplied, the unique part of the query is selected, NCS is applied and then the same analysis is carried out.

If desired, you can have chain_comparison write out all the residues in the query model that are matching residues in the target model.

Examples

Standard run of chain_comparison:

Running chain_comparison is easy. From the command-line you can type:

```
phenix.chain_comparison target.pdb query.pdb
```

If you want to compare P positions for RNA/DNA instead, you can say:

```
phenix.chain_comparison target.pdb query.pdb chain_type=RNA
```

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
 input_files_pdb_in = None Input PDB file (enter target first and then query)
 query_dir is set
 unique_query_only = False Use only unique chains in query. Normally use unique_query_only=False and unique_part_of_target_only=True.
 unique_target_pdb_in = None Target model identifying which element is selected with unique_query_only. NOTE: must be specified by keyword.
 unique_part_of_target_only = None Use only unique chains in target (see also unique_query_only).
 short_caption = Unique target only
 test_unique_part_of_target_only = True Try both unique_part_of_target_only as True and False and report result for whichever gives higher value of fraction matching. Cannot be used with match_pdb_file.
 allow_extensions = False If True, ignore parts of chains that do not overlap. Normally use False: identity is identity of overlapping part times fraction of chain that overlaps.
 ncs_file = None NCS file. If unique_query_only is False (typically) apply NCS to it to generate full query. Normally used with test_unique_part_of_target_only=True. NOTE: if your structure has very high symmetry, including an NCS file can result in extremely long run times. It may be better in such cases to supply target and query files that have NCS applied (or matching structures without NCS) and not to supply an NCS file.
 query_dir = None directory containing query PDB files (any number)
 output_files_match_pdb_file = None Output file containing segments with specified match percentage
 crystal_info_chain_type = *PROTEIN RNA DNA Chain type. All residues of other chain types ignored.
 use_crystal_symmetry = None Default is True if space group is not P1. If set, use crystal symmetry to map atoms to closest positions
 comparison_max_dist = 3. Maximum distance between atoms to be considered closed
 distance_per_site = None Maximum distance spanned by a pair of residues. Set by default as 3.8 Å for protein and 8 Å for RNA
 min_similarity = 0.99 When choosing unique chains, use min_similarity cutoff. This applies to both chain length and the sequence itself.
 target_length_from_matching_chains = False Use length of chains in target that are matched to define full target length (as opposed to all unique chains in target).
 minimum_percent_match_to_select = 1 You can specify minimum_percent_match_to_select and maximum_percent_match_to_select and match_pdb_file in which case all segments in the query model that have a percentage match (within max_dist of atom in target) in this range will be written out to match_pdb_file.
 maximum_percent_match_to_select = 100 You can specify minimum_percent_match_to_select and maximum_percent_match_to_select and match_pdb_file in which case all segments in the query model that have a percentage match (within max_dist of atom in target) in this range will be written out to match_pdb_file.
 remove_alt_conf = True Remove alternate conformers before analysis. This is normally required to align correctly.
 residue_groups = "VGASCTI P LDNEQM KR FHY W" Optional groups of residues to score together
 score_by_residue_groups = False Use residue groups in sequence alignment
 only_keep_best_chain = False Use keep the best-matching chain in match_pdb
 control_verbose = False Verbose output
 quiet = False No printed output
 gui GUI-specific parameter required for output directory
 output_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- input_files_pdb_in = None Input PDB file (enter target first and then query)
 query_dir is set
 unique_query_only = False Use only unique chains in query. Normally use unique_query_only=False and unique_part_of_target_only=True.
 unique_target_pdb_in = None Target model identifying which element is selected with unique_query_only. NOTE: must be specified by keyword.
 unique_part_of_target_only = None Use only unique chains in target (see also unique_query_only).
 short_caption = Unique target only
 test_unique_part_of_target_only = True Try both unique_part_of_target_only as True and False and report result for whichever gives higher value of fraction matching. Cannot be used with match_pdb_file.
 allow_extensions = False If True, ignore parts of chains that do not overlap. Normally use False: identity is identity of overlapping part times fraction of chain that overlaps.
 ncs_file = None NCS file. If unique_query_only is False (typically) apply NCS to it to generate full query. Normally used with test_unique_part_of_target_only=True. NOTE: if your structure has very high symmetry, including an NCS file can result in extremely long run times. It may be better in such cases to supply target and query files that have NCS applied (or matching structures without NCS)

and not to supply an NCS file.query_dir = None directory containing query PDB files (any number)

- pdb_in = None Input PDB file (enter target first and then query) query_dir is set)
- unique_query_only = False Use only unique chains in query. Normally use unique_query_only=False and unique_part_of_target_only=True.
- unique_target_pdb_in = None Target model identifying which element is selected with unique_query_only. NOTE: must be specified by keyword.
- unique_part_of_target_only = None Use only unique chains in target (see also unique_query_only).
.short_caption = Unique target only
- test_unique_part_of_target_only = True Try both unique_part_of_target_only as True and False and report result for whichever gives higher value of fraction matching. Cannot be used with match_pdb_file
- allow_extensions = False If True, ignore parts of chains that do not overlap. Normally use False: identity is identity of overlapping part times fraction of chain that overlaps.
- ncs_file = None NCS file. If unique_query_only is False (typically) apply NCS to it to generate full query. Normally used with test_unique_part_of_target_only=True. NOTE: if your structure has very high symmetry, including an NCS file can result in extremely long run times. It may be better in such cases to supply target and query files that have NCS applied (or matching structures without NCS) and not to supply an NCS file.
- query_dir = None directory containing query PDB files (any number)
- output_filesmatch_pdb_file = None Output file containing segments with specified match percentage
- match_pdb_file = None Output file containing segments with specified match percentage
- crystal_infochain_type = *PROTEIN RNA DNA Chain type. All residues of other chain types ignored.use_crystal_symmetry = None Default is True if space group is not P1. If set, use crystal symmetry to map atoms to closest positions
- chain_type = *PROTEIN RNA DNA Chain type. All residues of other chain types ignored.
- use_crystal_symmetry = None Default is True if space group is not P1. If set, use crystal symmetry to map atoms to closest positions
- comparisonmax_dist = 3. Maximum distance between atoms to be considered closedistance_per_site = None Maximum distance spanned by a pair of residues. Set by default as 3.8 A for protein and 8 A for RNAMin_similarity = 0.99 When choosing unique chains, use min_similarity cutoff. This applies to both chain length and the sequence itself.target_length_from_matching_chains = False Use length of chains in target that are matched to define full target length (as opposed to all unique chains in target).minimum_percent_match_to_select = 1 You can specify minimum_percent_match_to_select and maximum_percent_match_to_select and match_pdb_file in which case all segments in the query model that have a percentage match (within max_dist of atom in target) in this range will be written out to match_pdb_file.maximum_percent_match_to_select = 100 You can specify minimum_percent_match_to_select and maximum_percent_match_to_select and match_pdb_file in which case all segments in the query model that have a percentage match (within max_dist of atom in target) in this range will be written out to match_pdb_file.remove_alt_conf = True Remove alternate conformers before analysis. This is normally required to align correctly.residue_groups = "VGASCTI P LDNEQM KR FHY W" Optional groups of residues to score togetherscore_by_residue_groups = False Use residue groups in sequence alignmentonly_keep_best_chain = False Use keep the best-matching chain in match_pdb
- max_dist = 3. Maximum distance between atoms to be considered close

- `distance_per_site = None` Maximum distance spanned by a pair of residues. Set by default as 3.8 Å for protein and 8 Å for RNA
- `min_similarity = 0.99` When choosing unique chains, use `min_similarity` cutoff. This applies to both chain length and the sequence itself.
- `target_length_from_matching_chains = False` Use length of chains in target that are matched to define full target length (as opposed to all unique chains in target).
- `minimum_percent_match_to_select = 1` You can specify `minimum_percent_match_to_select` and `maximum_percent_match_to_select` and `match_pdb_file` in which case all segments in the query model that have a percentage match (within `max_dist` of atom in target) in this range will be written out to `match_pdb_file`.
- `maximum_percent_match_to_select = 100` You can specify `minimum_percent_match_to_select` and `maximum_percent_match_to_select` and `match_pdb_file` in which case all segments in the query model that have a percentage match (within `max_dist` of atom in target) in this range will be written out to `match_pdb_file`.
- `remove_alt_conf = True` Remove alternate conformers before analysis. This is normally required to align correctly.
- `residue_groups = "VGASCTI P LDNEQM KR FHY W"` Optional groups of residues to score together
- `score_by_residue_groups = False` Use residue groups in sequence alignment
- `only_keep_best_chain = False` Use keep the best-matching chain in `match_pdb`
- `controlverbose = False` Verbose output `quiet = False` No printed output
- `verbose = False` Verbose output
- `quiet = False` No printed output
- `gui` GUI-specific parameter required for output directory `output_dir = None`
- `output_dir = None`

<string>

model_vs_data.html

Inputs:

File with reflection data (Fobs or lobs), and R-free flags (optionally); label(s) selecting which reflection data arrays should be used (in case there are multiple choices in input file, there is no need to provide labels otherwise); PDB file with input model; some other optional parameters.

- File with reflection data (Fobs or lobs), and R-free flags (optionally);
- label(s) selecting which reflection data arrays should be used (in case there are multiple choices in input file, there is no need to provide labels otherwise);
- PDB file with input model;
- some other optional parameters.

Usage examples:

```
phenix.model_vs_data model.pdb data.hkl phenix.model_vs_data model.pdb data.hkl f_obs_label="F"
r_free_flags_label="FREE" phenix.model_vs_data model.pdb data.hkl scattering_table=neutron
phenix.model_vs_data model.pdb data.hkl twin_law='h,-k,l+h'
```

- phenix.model_vs_data model.pdb data.hkl
- phenix.model_vs_data model.pdb data.hkl f_obs_label="F" r_free_flags_label="FREE"
- phenix.model_vs_data model.pdb data.hkl scattering_table=neutron
- phenix.model_vs_data model.pdb data.hkl twin_law='h,-k,l+h'

Default parameters: f_obs_label = None r_free_flags_label = None scattering_table = wk1995 it1992
*n_gaussian neutron high_resolution = None twin_law = None n_bins = 20

- f_obs_label = None
- r_free_flags_label = None
- scattering_table = wk1995 it1992 *n_gaussian neutron
- high_resolution = None
- twin_law = None
- n_bins = 20

Morphing a model with morph_model

morph_model.html

Contents

- Author(s)
- Purpose
- Summary of the procedure used in morph_model
- Maps for morph_model
- Standard run of morph_model
- Supplying a map for morph_model
- Morphing just part of a model
- Getting a default parameters file for morph_model
- Possible Problems
- Specific limitations and problems
- Literature
- Additional information
- List of all available keywords

Author(s)

- morph_model: Tom Terwilliger

Purpose

morph_model is a procedure for distorting a model to match an electron density map. It is suited for a case where a poor but unbiased map is available and a model that is generally locally similar to the target structure but where longer-range distances are not conserved. This is often the case in homologous structures.

Summary of the procedure used in morph_model

The basic idea in morph_model is to apply a smoothly-varying offset to the template structure so as to make it match the map as closely as possible. In this way information about the local structure of the template is retained while allowing longer-range changes in structure.

Morphing is carried out in 3 steps:

- For each residue in the structure an offset is identified that optimizes the match of the map to a sphere of atoms within rad_morph (typically 6 Å) of the CA atom of that residue.
- The offsets are smoothed in a window of 10 residues
- The smoothed offset for each residue is applied to all the atoms in that residue

After morphing, the morphed model is refined. This process is then repeated morph_cycles (typically 6) times.

Maps for morph_model

Normally you should specify that morph-model use a prime-and-switch map (map_type=prime_and_switch), as that is usually the most effective map for morphing. You can however choose instead to use a 2mFo-DFc map, a density-modified map, or an omit map. The prime-and-switch map is a map calculated in the same way as a density-modified map, except that once the initial model-based map is calculated (priming), the model information is no longer used (switching). This map is typically quite unbiased by the model.

Standard run of morph_model

Running morph_model is easy. If you have a template (coords1.pdb) and a data file fobs.mtz with FP SIGFP and FreeR_flag, you can type:

```
phenix.morph_model data=coords1.mtz model=coords1.pdb
```

and morph_model will run automatically.

Supplying a map for morph_model

If you have your own map you want to use in morph_model, you can type:

```
phenix.morph_model data=coords1.mtz \  
model=coords1.pdb map_coeffs=my_map_coeffs.mtz
```

and morph_model will start off with your map coefficients instead of calculating a new map. If you specify update_map=True (default) then new maps will be calculated on each subsequent cycle. If you specify update_map=False then your map will be used throughout.

Morphing just part of a model

You can morph just one part of your model if you want. You can specify it like this:

```
phenix.morph_model data=coords1.mtz \  
model=coords1.pdb morph_selection="chain A"
```

The remainder of the model will be refined but not morphed.

Getting a default parameters file for morph_model

Usually you will want to edit a parameters file so that you can specify more details of the run. You can get a default parameters file with:

```
phenix.morph_model
```

and then just edit the file morph_model_params.dat.

Possible Problems

Specific limitations and problems

Literature

Improved crystallographic models through iterated local density-guided model deformation and reciprocal-space refinement. T.C. Terwilliger, R.J. Read, P.D. Adams, A.T. Brunger, P.V. Afonine, R.W. Grosse-Kunstleve, and L.W. Hung. Acta Crystallogr D Biol Crystallogr 68, 861-70 (2012).

- Improved crystallographic models through iterated local density-guided model deformation and reciprocal-space refinement. T.C. Terwilliger, R.J. Read, P.D. Adams, A.T. Brunger, P.V. Afonine, R.W. Grosse-Kunstleve, and L.W. Hung. *Acta Crystallogr D Biol Crystallogr* 68, 861-70 (2012).

Additional information

List of all available keywords

input_filesdata = None Data file with experimental data (FP SIGFP or I SIGI).data_labels = None Optional labels for experimental data Normally these would be something like I,SIGI or F,SIGFfree_r_data = None Optional data file with free_r flags. By default this is the same file as used for experimental data.free_r_labels = None Optional labels for free_r flags. Normally these would be something like FreeR_flags or R_free_flagshl_data = None Optional data file with hl coeffs. By default this is the same file as used for experimental data.hl_labels = None Optional labels for hl flags. Normally these would be something like HLA,HLB,HLC,HLDlabin = None Optional Labin line for file with data. This is present for backward compatibility only. Normally use data_labels instead. Only allowed if the data file contains myFP mySIGF and myFreeR_flag: LABIN FP=myFP SIGFP=mySIGFP FreeR_flag=myFreeR_flagmodel = None Input PDB file with coordinates to be refined and morphed.map_coeffs = None Data file (mtz format) with map coeffs (FP PHIB FOM or FWT PHWT) for use in morphing. Normally use None. If set, then these map coefficients will be used instead of an omit map.labin_map_coeffs = None Labin line for map coeffs file Something like FP=FP PHIB=PHIM FOM=FOMM Normally use None; used by morph_model in iterationoutputtarget_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.output_fileslog = morph_model.log Output log filemorphed_model = morphed_model.pdb Output morphed modelparams_out = morph_model_params.eff Parameters file to rerun morph_modeldirectoriestemp_dir = "" Optional temporary work directory workdir = "" Optional work directory. Base path for all work output_dir = "" Output directory where files are to be writtentop_output_dir = None Output directory for entire set of runsgui_output_dir = None Base directory for PHENIX GUI. Ignored if running the command-line version.crystal_infospace_group = None Optional space group specificationresolution = 0. high-resolution limit for map calculationsolvent_fraction = None Optional solvent fractionchain_type = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms)correct_aniso = False Remove anisotropy from data final_b = None Overall B of aniso-corrected data. If None, then minimum of the diagonal of anisotropic B values is usedmorphingnumber_of_cycles = 6 Number of cycles of morphing rad_morph = 6 Radius for morphingrad_morph_final = None Optional final radius for morphing. If set, rad_morph will be changed from initial value to final value over the cycles of morphingmorph_main = False You can choose whether to use only main-chain atoms plus c-beta atoms in calculation of shifts in morphing. Default is morph_main=False; use all atoms including side-chain atoms.morph_selection = None You can specify an atom selection identifying which parts of your molecule to morph. These should be contiguous segments such as a chain or some segments in a chain. The remainder will be refined only.map_type = simple_omit refine_omit 2fofc density_modified *prime_and_switch Map type to use in morphing. Ignored if map_coeffs is setuse_ncs_in_ps = False Use NCS (if available) in prime-and-switch mapsupdate_map = True Create new map for morphing at each cycle. Ignored if map_coeffs is setrefinementrefinement_params = None Input parameters file for phenix.refine skip_clash_guard = True Skip clash guard check in refinementcorrect_special_position_tolerance = None Adjust tolerance for special position check. If 0., then check for clashes near special positions is not carried out. This sometimes allows phenix.refine to continue even if an atom is near a special position. If 1., then checks within 1 A of special positions. If None, then uses phenix.refine default. (1)reference_restraints = False Restrain angles to starting model. This may be u...

- `input_filesdata` = None Data file with experimental data (FP SIGFP or I SIGI).`data_labels` = None
Optional labels for experimental data Normally these would be something like I,SIGI or F,SIGF
- `free_r_data` = None Optional data file with `free_r` flags. By default this is the same file as used for experimental data.`free_r_labels` = None Optional labels for `free_r` flags. Normally these would be something like `FreeR_flags` or `R_free_flags`
- `hl_data` = None Optional data file with `hl` coeffs. By default this is the same file as used for experimental data.`hl_labels` = None Optional labels for `hl` flags. Normally these would be something like HLA,HLB,HLC,HLD
- `labin` = None Optional Labin line for file with data. This is present for backward compatibility only. Normally use `data_labels` instead. Only allowed if the data file contains myFP mySIGF and myFreeR_flag: LABIN FP=myFP SIGFP=mySIGFP FreeR_flag=myFreeR_flag
- `model` = None Input PDB file with coordinates to be refined and morphed.
- `map_coeffs` = None Data file (mtz format) with map coeffs (FP PHIB FOM or FWT PHWT) for use in morphing. Normally use None. If set, then these map coefficients will be used instead of an omit map.
- `labin_map_coeffs` = None Labin line for map coeffs file Something like FP=FP PHIB=PHIM FOM=FOMM Normally use None; used by `morph_model` in iteration
- `outputtarget_output_format` = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `target_output_format` = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `output_fileslog` = `morph_model.log` Output log file
`morphed_model` = `morphed_model.pdb` Output morphed model
`params_out` = `morph_model_params.eff` Parameters file to rerun `morph_model`
- `log` = `morph_model.log` Output log file
- `morphed_model` = `morphed_model.pdb` Output morphed model

- `params_out = morph_model_params.eff` Parameters file to rerun `morph_model`
- `directories`
`temp_dir = ""` Optional temporary work directory
`workdir = ""` Optional work directory. Base path for all work
`output_dir = ""` Output directory where files are to be written
`top_output_dir = None` Output directory for entire set of runs
`gui_output_dir = None` Base directory for PHENIX GUI. Ignored if running the command-line version.
- `temp_dir = ""` Optional temporary work directory
- `workdir = ""` Optional work directory. Base path for all work
- `output_dir = ""` Output directory where files are to be written
- `top_output_dir = None` Output directory for entire set of runs
- `gui_output_dir = None` Base directory for PHENIX GUI. Ignored if running the command-line version.
- `crystal_info`
`space_group = None` Optional space group specification
`resolution = 0.` high-resolution limit for map calculation
`solvent_fraction = None` Optional solvent fraction
`chain_type = *PROTEIN DNA RNA` Chain type (for identifying main-chain and side-chain atoms)
`correct_aniso = False` Remove anisotropy from data
`final_b = None` Overall B of aniso-corrected data. If None, then minimum of the diagonal of anisotropic B values is used
- `space_group = None` Optional space group specification
- `resolution = 0.` high-resolution limit for map calculation
- `solvent_fraction = None` Optional solvent fraction
- `chain_type = *PROTEIN DNA RNA` Chain type (for identifying main-chain and side-chain atoms)
- `correct_aniso = False` Remove anisotropy from data
- `final_b = None` Overall B of aniso-corrected data. If None, then minimum of the diagonal of anisotropic B values is used
- `morphing`
`number_of_cycles = 6` Number of cycles of morphing
`rad_morph = 6` Radius for morphing
`rad_morph_final = None` Optional final radius for morphing. If set, `rad_morph` will be changed from initial value to final value over the cycles of morphing
`morph_main = False` You can choose whether to use only main-chain atoms plus c-beta atoms in calculation of shifts in morphing. Default is `morph_main=False`; use all atoms including side-chain atoms.
`morph_selection = None` You can specify an atom selection identifying which parts of your molecule to morph. These should be contiguous segments such as a chain or some segments in a chain. The remainder will be refined only.
`map_type = simple` Map type to use in morphing. Ignored if `map_coeffs` is set
`use_ncs_in_ps = False` Use NCS (if available) in prime-and-switch maps
`update_map = True` Create new map for morphing at each cycle. Ignored if `map_coeffs` is set
- `number_of_cycles = 6` Number of cycles of morphing
- `rad_morph = 6` Radius for morphing
- `rad_morph_final = None` Optional final radius for morphing. If set, `rad_morph` will be changed from initial value to final value over the cycles of morphing
- `morph_main = False` You can choose whether to use only main-chain atoms plus c-beta atoms in calculation of shifts in morphing. Default is `morph_main=False`; use all atoms including side-chain atoms.
- `morph_selection = None` You can specify an atom selection identifying which parts of your molecule to morph. These should be contiguous segments such as a chain or some segments in a chain. The remainder will be refined only.

- `map_type = simple_omit_refine_omit_2fofc_density_modified_prime_and_switch` Map type to use in morphing. Ignored if `map_coeffs` is set
- `use_ncs_in_ps = False` Use NCS (if available) in prime-and-switch maps
- `update_map = True` Create new map for morphing at each cycle. Ignored if `map_coeffs` is set
- `refinement_refinement_params = None` Input parameters file for phenix.refine `skip_clash_guard = True` Skip clash guard check in refinement `correct_special_position_tolerance = None` Adjust tolerance for special position check. If 0., then check for clashes near special positions is not carried out. This sometimes allows phenix.refine to continue even if an atom is near a special position. If 1., then checks within 1 Å of special positions. If None, then uses phenix.refine default. (1) `reference_restraints = False` Restrain angles to starting model. This may be useful at lower resolution. `use_hl = None` You can choose to use HL coefficients in refinement. If None, hl coeffs will be used if available. `test_flag_value = None` You can set the test flag value if you want
- `refinement_params = None` Input parameters file for phenix.refine
- `skip_clash_guard = True` Skip clash guard check in refinement
- `correct_special_position_tolerance = None` Adjust tolerance for special position check. If 0., then check for clashes near special positions is not carried out. This sometimes allows phenix.refine to continue even if an atom is near a special position. If 1., then checks within 1 Å of special positions. If None, then uses phenix.refine default. (1)
- `reference_restraints = False` Restrain angles to starting model. This may be useful at lower resolution.
- `use_hl = None` You can choose to use HL coefficients in refinement. If None, hl coeffs will be used if available
- `test_flag_value = None` You can set the test flag value if you want
- `control_verbose = False` Verbose output `debug = False` Debugging output `raise_sorry = False` Raise sorry if problems `dry_run = False` Just read in and check parameter names `nproc = 1` Number of processors to use `group_run_command = "sh "` Not currently enabled. Command to use to run multiple jobs. This may be sh if you are using a single machine (where you might set `background=True`) or something like 'qsub' or 'qsub -q all.q@theta' on a cluster (where you should leave `background=False`) `queue_commands = None` Not currently enabled. You can add any commands that need to be run for your queueing system. For example on a PBS system you might say: `queue_commands='#PBS -N morph_model'` `queue_commands='#PBS -j oe'` `queue_commands='#PBS -l walltime=03:00:00'` `queue_commands='#PBS -l nodes=1:ppn=1'` NOTE: you can put in the characters '<path>' in any `queue_commands` line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_', up to 15 characters. For example 'test_autobuild/WORK_5/AutoBuild_run_1/TEMPO/RUN_1' would be represented by: 'tld_W_5_A1__TPO_1' `condor_universe = vanilla` The universe for condor is usually vanilla. However you might need to set it to local for your cluster `add_double_quotes_in_condor = True` You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths. `condor = None` Not currently enabled. Specifies if the `group_run_command` is submitting a job to a condor cluster. Set by default to True if `group_run_command=condor_submit`, otherwise False. For condor job submission `morph_model` uses a customized script with condor commands. Also uses `one_subprocess_level=True` `one_subprocess_level = None` Specifies that a subprocess cannot submit a job `single_run_command = "sh "` Command to use to run single jobs. Normally this is `shbackground = None` Run in background. If None, automatically set to True if `nproc` is greater than one and `group_run_command` is `shignore_errors_in_subprocess = True` Generally use `ignore_errors_in_subprocess=True` to ignore errors in sub-processes. This allows you to

continue even if a few jobs crash. If all jobs in a group crash, the process will stop. NOTE: if a job hangs or never runs...this will not be detected and you will have to either put a file with the name FINISHED in the directory where the job was to run (e.g, MORPH_MODEL_3/FINISHED) or stop the whole job by putting a file with the name STOPWIZARD in the main run directory (e.g., MORPH_MODEL_3/STOPWIZARD) `check_run_command = False` Try out run command to make sure it works Use False if your queue may not be available at the beginning of your run. Use True if you want to check things out `max_wait_time = 100` Maximum time (sec) to wait for a file to be written (Useful for queues or nfs-mounted systems) `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is File exists:.... `wizard_directory_number = None` Directory number for MORPH_MODEL_xx. Normally None except if called from GUI `dir_max = 100000` Maximum number of directories to create (must be as big as `nproc` or `nstruct/chunk`) `write_run_directory_to_file = None` The working directory name is written to this file `job_title = None` Job title in PHENIX GUI, not used on command line

- `verbose = False` Verbose output
- `debug = False` Debugging output
- `raise_sorry = False` Raise sorry if problems
- `dry_run = False` Just read in and check parameter names
- `nproc = 1` Number of processors to use
- `group_run_command = "sh "` Not currently enabled. Command to use to run multiple jobs This may be `sh` if you are using a single machine (where you might set `background=True`) or something like `'qsub'` or `'qsub -q all.q@theta'` on a cluster (where you should leave `background=False`)
- `queue_commands = None` Not currently enabled. You can add any commands that need to be run for your queueing system. For example on a PBS system you might say: `queue_commands='#PBS -N morph_model'` `queue_commands='#PBS -j oe'` `queue_commands='#PBS -l walltime=03:00:00'` `queue_commands='#PBS -l nodes=1:ppn=1'` NOTE: you can put in the characters '<path>' in any `queue_commands` line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_', up to 15 characters. For example `'test_autobuild/WORK_5/AutoBuild_run_1/TEMP0/RUN_1'` would be represented by: `'tld_W_5_A1__TP0_1'`
- `condor_universe = vanilla` The universe for condor is usually vanilla. However you might need to set it to local for your cluster
- `add_double_quotes_in_condor = True` You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.
- `condor = None` Not currently enabled. Specifies if the `group_run_command` is submitting a job to a condor cluster. Set by default to True if `group_run_command=condor_submit`, otherwise False. For condor job submission `morph_model` uses a customized script with condor commands. Also uses `one_subprocess_level=True`
- `one_subprocess_level = None` Specifies that a subprocess cannot submit a job
- `single_run_command = "sh "` Command to use to run single jobs Normally this is `sh`
- `background = None` Run in background. If None, automatically set to True if `nproc` is greater than one and `group_run_command` is `sh`

- `ignore_errors_in_subprocess = True` Generally use `ignore_errors_in_subprocess=True` to ignore errors in sub-processes. This allows you to continue even if a few jobs crash. If all jobs in a group crash, the process will stop. NOTE: if a job hangs or never runs...this will not be detected and you will have to either put a file with the name `FINISHED` in the directory where the job was to run (e.g., `MORPH_MODEL_3/FINISHED`) or stop the whole job by putting a file with the name `STOPWIZARD` in the main run directory (e.g., `MORPH_MODEL_3/STOPWIZARD`)
- `check_run_command = False` Try out run command to make sure it works Use `False` if your queue may not be available at the beginning of your run. Use `True` if you want to check things out
- `max_wait_time = 100` Maximum time (sec) to wait for a file to be written (Useful for queues or nfs-mounted systems)
- `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is `File exists:....`
- `wizard_directory_number = None` Directory number for `MORPH_MODEL_xx`. Normally `None` except if called from GUI
- `n_dir_max = 100000` Maximum number of directories to create (must be as big as `nproc` or `nstruct/chunk`)
- `write_run_directory_to_file = None` The working directory name is written to this file
- `job_title = None` Job title in PHENIX GUI, not used on command line
- `non_user_paramsprint_citations = True` Print citation information at end of `runis_sub_process = False` identifies if this is a sub-process or top-level job
- `print_citations = True` Print citation information at end of run
- `is_sub_process = False` identifies if this is a sub-process or top-level job

phenix.pdbtools: PDB model manipulations and statistics

[pdbtools.html](#)

Contents

- Manipulations on a model in a PDB file including
- Video Tutorial
- Graphical interface
- Command-line use
- List of all available keywords

Manipulations on a model in a PDB file including

The operations below can be applied to the whole model or selected parts (e.g. "modify.selection=chain A and backbone"). See examples below.

- shaking of coordinates (random coordinate shifts)
- rotation-translation shift of coordinates
- shaking of occupancies
- set occupancies to a value
- shaking of ADP (B-factors)
- shifting of ADP (addition of a constant value)
- scaling of ADP (multiplication by a constant value)
- setting ADP to a given value
- conversion to isotropic or anisotropic ADP
- removal of selected parts of a model
- renumbering of residues
- truncation to poly-ALA
- convert SeMet residues to Met
- adjust segid field in PDB format

Video Tutorial

The tutorial video is available on the Phenix YouTube channel and covers the following topics:

- basic overview
- how to run PDBTools via the GUI
- example 1: prepare model file for usage with lower resolution data
- example 2: convert all residues in a model to alanine

Graphical interface

As input, only a PDB file and appropriate geometry restraints (in CIF format) are required. (The geometry restraints are actually optional if you uncheck the box "Stop for unknown residues", but they are strongly

recommended.) Crystal symmetry is not essential but also recommended, and will be propagated to the output model.

Many different options are possible, only some of which are shown in the main window. The atom selection controls can be used to specify which part of the model will be affected by various changes, or to eliminate parts of the model. The operations on coordinates, B-factors, and occupancies also have separate (optional) atom selections.

(Note: in past versions PDBTools was also responsible for geometry minimization and addition of hydrogen atoms using Reduce; these methods have been moved to the programs `phenix.geometry_minimization` and `phenix.ready_set`, respectively.)

Command-line use

Some specific examples:

1) Type `phenix.pdbtools` from the command line for instructions:

```
% phenix.pdbtools
```

2) To see all default parameters:

```
% phenix.pdbtools
```

3) Suppose a PDB model consist of three chains A, B and C and some water molecules. Remove all atoms in chain C and all waters:

```
% phenix.pdbtools model.pdb remove="chain C or water"
```

or one can achieve exactly the same result with equivalent command:

```
% phenix.pdbtools model.pdb keep="chain A or chain B"
```

or:

```
% phenix.pdbtools model.pdb keep="not(chain C or water)"
```

or finally:

```
% phenix.pdbtools model.pdb remove="not(chain A or chain B)"
```

The result of all four equivalent commands above will be a new PDB file containing chains A and B only. Important: the commands `keep` and `remove` cannot be used simultaneously.

4) Remove all water and set all b-factors to 25:

```
% phenix.pdbtools model.pdb remove=water set_b_iso=25
```

5) Suppose a PDB model consist of three chains A, B and C and some water molecules. Remove all but water atoms and set b-factors to 25 for chain C atoms:

```
% phenix.pdbtools model.pdb keep=water set_b_iso=25 modify.selection="chain C"
```

List of all available keywords

`modifyremove` = None Selection for the atoms to be removed
`keepput_into_box_with_buffer` = None Move molecule into center of box
`selection` = None Selection for atoms to be modified
`flip_symmetric_amino_acids` = False Flip symmetric amino acid side chains
`change_of_basis` = None Apply change-of-basis operator (e.g. reindexing operator) to the coordinates and symmetry. Example: `a,c,b`
`renumber_residues` = False Re-number residues
`increment_resseq` = None Increment residue number
`truncate_to_polyala` = False Truncate a model to poly-Ala.
`truncate_to_polygly` = False Truncate a

model to poly-Gly.remove_alt_confs = False Deletes atoms whose altloc identifier is not blank or A , and resets the occupancies of the remaining atoms to 1.0.always_keep_one_conformer = False Modifies behavior of remove_alt_confs so that residues with no conformer labeled blank or A are not deleted. Silent if remove_alt_confs is False.keep_occupancy = False Do not reset occupancy to 1 after removing altlocsaltloc_to_keep = None Modifies behavior of remove_alt_confs so that the altloc identifier to keep in addition to blank is this one (instead of A). Silent if remove_alt_confs is False.average_alt_confs = False Averages the coordinates of altloc atoms with 1.0 angstrom.set_chemical_element_simple_if_necessary = None Make a simple guess about what the chemical element is (based on atom name and the way how it is formatted) and write it into output file.set_seg_id_to_chain_id = False Sets the segID field to the chain ID (padded with spaces).clear_seg_id = False Erases the segID field.convert_somet_to_met = False convert_met_to_somet = False neutralize_scatterers = False remove_fraction = None random_seed = None Random seedmove_waters_last = False Transfer waters to the end of the model. Addresses some limitations of water picking in phenix.refine.renumber_and_move_waters = False Transfer waters to the end of respective chains. Renumber them so that numbering starts at the nearest 100 after the last non-water in the chain.adpScope of options to modify ADP of selected atomsatom_selection = None Selection for atoms to be modified. Overrides parent-level selection.randomize = False Randomize ADP within a certain rangeset_b_iso = None Set ADP of atoms to set_b_isoconvert_to_isotropic = False Convert atoms to isotropicconvert_to_anisotropic = False Convert atoms to anisotropicshift_b_iso = None Add shift_b_iso value to ADPscale_adp = None Multiply ADP by scale_adpsitesScope of options to modify coordinates of selected atomsatom_selection = None Selection for atoms to be modified. Overrides parent-level selection.shake = None Randomize coordinates with mean error value equal to shakewswitch_rotamers = max_distant min_distant exact_match fix_outliers translate = 0 0 0 Translational shift (x,y,z)rotate = 0 0 0 Rotational shift (x,y,z)euler_angle_convention = *xyz zyz Euler angles convention to be used for rotationoccupanciesScope of options to modify occupancies of selected atomsatom_selection = None Selection for atoms to be modified. Overrides parent-level selection.randomize = False Randomize occupancies within a certain rangeset = None Set all or selected occupancies to given valuerotate_about_axisaxis = None angle = None atom_selection = None rename_chain_idRename chainsold_id = None new_id = None set_chargecharge_selection = None charge = None

- modifyremove = None Selection for the atoms to be removedkeep = None Select atoms to keepput_into_box_with_buffer = None Move molecule into center of boxselection = None Selection for atoms to be modifiedflip_symmetric_amino_acids = False Flip symmetric amino acid side chainschange_of_basis = None Apply change-of-basis operator (e.g. reindexing operator) to the coordinates and symmetry. Example: a,c,b .renumber_residues = False Re-number residuesincrement_resseq = None Increment residue numbertruncate_to_polyala = False Truncate a model to poly-Ala.truncate_to_polygly = False Truncate a model to poly-Gly.remove_alt_confs = False Deletes atoms whose altloc identifier is not blank or A , and resets the occupancies of the remaining atoms to 1.0.always_keep_one_conformer = False Modifies behavior of remove_alt_confs so that residues with no conformer labeled blank or A are not deleted. Silent if remove_alt_confs is False.keep_occupancy = False Do not reset occupancy to 1 after removing altlocsaltloc_to_keep = None Modifies behavior of remove_alt_confs so that the altloc identifier to keep in addition to blank is this one (instead of A). Silent if remove_alt_confs is False.average_alt_confs = False Averages the coordinates of altloc atoms with 1.0 angstrom.set_chemical_element_simple_if_necessary = None Make a simple guess about what the chemical element is (based on atom name and the way how it is formatted) and write it into output file.set_seg_id_to_chain_id = False Sets the segID field to the chain ID (padded with spaces).clear_seg_id = False Erases the segID field.convert_somet_to_met = False convert_met_to_somet = False neutralize_scatterers = False remove_fraction = None random_seed = None Random seedmove_waters_last = False Transfer waters to the end of the model. Addresses some limitations of water picking in phenix.refine.renumber_and_move_waters = False Transfer waters to the

end of respective chains. Renumber them so that numbering starts at the nearest 100 after the last non-water in the chain.

adpScope of options to modify ADP of selected atoms
 atom_selection = None Selection for atoms to be modified. Overrides parent-level selection.
 randomize = False Randomize ADP within a certain range
 set_b_iso = None Set ADP of atoms to set_b_iso
 convert_to_isotropic = False Convert atoms to isotropic
 convert_to_anisotropic = False Convert atoms to anisotropic
 shift_b_iso = None Add shift_b_iso value to ADP
 scale_adp = None Multiply ADP by scale_adp
 sitesScope of options to modify coordinates of selected atoms
 atom_selection = None Selection for atoms to be modified. Overrides parent-level selection.
 shake = None Randomize coordinates with mean error value equal to shake
 switch_rotamers = max_distant min_distant exact_match fix_outliers translate = 0 0 0 Translational shift (x,y,z)
 rotate = 0 0 0 Rotational shift (x,y,z)
 euler_angle_convention = *xyz zyz Euler angles convention to be used for rotation
 occupanciesScope of options to modify occupancies of selected atoms
 atom_selection = None Selection for atoms to be modified. Overrides parent-level selection.
 randomize = False Randomize occupancies within a certain range
 set = None Set all or selected occupancies to given value
 rotate_about_axis = None angle = None atom_selection = None
 rename_chain_id Rename chains
 old_id = None new_id = None set_charge
 charge_selection = None charge = None

- remove = None Selection for the atoms to be removed
- keep = None Select atoms to keep
- put_into_box_with_buffer = None Move molecule into center of box
- selection = None Selection for atoms to be modified
- flip_symmetric_amino_acids = False Flip symmetric amino acid side chains
- change_of_basis = None Apply change-of-basis operator (e.g. reindexing operator) to the coordinates and symmetry. Example: a,c,b .
- renumber_residues = False Re-number residues
- increment_resseq = None Increment residue number
- truncate_to_polyala = False Truncate a model to poly-Ala.
- truncate_to_polygly = False Truncate a model to poly-Gly.
- remove_alt_confs = False Deletes atoms whose altloc identifier is not blank or A , and resets the occupancies of the remaining atoms to 1.0.
- always_keep_one_conformer = False Modifies behavior of remove_alt_confs so that residues with no conformer labeled blank or A are not deleted. Silent if remove_alt_confs is False.
- keep_occupancy = False Do not reset occupancy to 1 after removing altlocs
- altloc_to_keep = None Modifies behavior of remove_alt_confs so that the altloc identifier to keep in addition to blank is this one (instead of A). Silent if remove_alt_confs is False.
- average_alt_confs = False Averages the coordinates of altloc atoms with 1.0 angstrom.
- set_chemical_element_simple_if_necessary = None Make a simple guess about what the chemical element is (based on atom name and the way how it is formatted) and write it into output file.
- set_seg_id_to_chain_id = False Sets the segID field to the chain ID (padded with spaces).
- clear_seg_id = False Erases the segID field.
- convert_semet_to_met = False
- convert_met_to_semet = False
- neutralize_scatterers = False

- `remove_fraction` = None
- `random_seed` = None Random seed
- `move_waters_last` = False Transfer waters to the end of the model. Addresses some limitations of water picking in `phenix.refine`.
- `renumber_and_move_waters` = False Transfer waters to the end of respective chains. Renumber them so that numbering starts at the nearest 100 after the last non-water in the chain.
- `adpScope` of options to modify ADP of selected atoms
`atom_selection` = None Selection for atoms to be modified. Overrides parent-level selection.
`randomize` = False Randomize ADP within a certain range
`set_b_iso` = None Set ADP of atoms to `set_b_iso`
`convert_to_isotropic` = False Convert atoms to isotropic
`convert_to_anisotropic` = False Convert atoms to anisotropic
`shift_b_iso` = None Add `shift_b_iso` value to ADP
`scale_adp` = None Multiply ADP by `scale_adp`
- `sitesScope` of options to modify coordinates of selected atoms
`atom_selection` = None Selection for atoms to be modified. Overrides parent-level selection.
`shake` = None Randomize coordinates with mean error value equal to `shake`
`switch_rotamers` = `max_distant` `min_distant` `exact_match` `fix_outliers` `translate` = 0 0 0 Translational shift (x,y,z)
`rotate` = 0 0 0 Rotational shift (x,y,z)
`euler_angle_convention` = *xyz zyz Euler angles convention to be used for rotation
- `atom_selection` = None Selection for atoms to be modified. Overrides parent-level selection.
- `shake` = None Randomize coordinates with mean error value equal to `shake`
- `switch_rotamers` = `max_distant` `min_distant` `exact_match` `fix_outliers`
- `translate` = 0 0 0 Translational shift (x,y,z)
- `rotate` = 0 0 0 Rotational shift (x,y,z)
- `euler_angle_convention` = *xyz zyz Euler angles convention to be used for rotation
- `occupanciesScope` of options to modify occupancies of selected atoms
`atom_selection` = None Selection for atoms to be modified. Overrides parent-level selection.
`randomize` = False Randomize occupancies within a certain range
`set` = None Set all or selected occupancies to given value
- `atom_selection` = None Selection for atoms to be modified. Overrides parent-level selection.
- `randomize` = False Randomize occupancies within a certain range
- `set` = None Set all or selected occupancies to given value
- `rotate_about_axis` `axis` = None `angle` = None `atom_selection` = None
- `axis` = None
- `angle` = None
- `atom_selection` = None
- `rename_chain_id` `old_id` = None `new_id` = None

- old_id = None
- new_id = None
- set_chargecharge_selection = None charge = None
- charge_selection = None
- charge = None

Processing AlphaFold2, RoseTTAFold and other predicted models

process_predicted_model.html

Author(s)

- process_predicted_model: Tom Terwilliger, Claudia Millan Nebot, Tristan Croll

Purpose

Process model files produced by AlphaFold, RoseTTAFold and other prediction software, replacing information these programs put in the B-factor field with pseudo-B values and optionally breaking the model into compact domains.

Background

As described in AlphaFold and Phenix, structure prediction software is now capable of generating models that are highly accurate over some or all parts of the models. Importantly, these predictions often come with reliable residue-by-residue estimates of uncertainty. The process_predicted_model tool is designed to help you remove low-confidence residues and break the model into domains for docking or molecular replacement.

How process_predicted_model works:

The process_predicted_model tool uses estimates of uncertainty supplied by structure prediction tools in the B-value (atomic displacement parameters) field of a model to create new pseudo-B values, to remove uncertain parts of the model, and to break up the model into domains.

If you used the Phenix server to get an AlphaFold prediction (See phenix.predict_model), the uncertainty estimate will be pLDDT (see section below for details of this metric).

The B-value field in most predicted models represents one of three possible values:

An actual B-value (atomic displacement parameter)

An estimate of error in A (rmsd)

A confidence (LDDT) on a scale of either 0 to 1 or 0 to 100.

In process_predicted_model, error estimates in A or confidence values are first converted to B-values. Then residues with high B-values are removed. Then the remaining residues are grouped (optionally) into domains.

Conversion of error estimates to B-values

Error estimates in A are converted to B-values using the standard formula for the relationship between rms positional variation and B-values:

$$B = \text{rmsd}^2 * ((8 * (\pi^2)) / 3.0)$$

Conversion of LDDT values to error estimates

LDDT values are first converted to a scale of 0 to 1. You can specify whether the LDDT values in your model are from 0 to 1 (fractional) or from 0 to 100. If you don't specify, a model with all LDDT

values between 0 and 1 is assumed to contain fractional LDDT values.

Then LDDT values on a scale of 0 to 1 are converted to error estimates using an empirical formula from

Hiranuma, N., Park, H., Baek, M. et al. Improved protein structure refinement guided by deep learning based accuracy estimation. Nat Commun 12, 1340 (2021). <https://doi.org/10.1038/s41467-021-21511-x>

Hiranuma, N., Park, H., Baek, M. et al. Improved protein structure

refinement guided by deep learning based accuracy estimation. Nat Commun 12, 1340 (2021).

<https://doi.org/10.1038/s41467-021-21511-x>

This empirical formula is:

$$\text{RMSD} = 1.5 * \exp(4 * (0.7 - \text{LDDT}))$$

Trimming away low-confidence regions from predicted models

Normally it is a good idea to remove low-confidence regions from a predicted model before using them as a starting point for experimental structure determination. For AlphaFold2 models, low-confidence corresponds approximately to an LDDT value of about 0.7 (on a scale of 0 to 1, or 70 on a scale of 0 to 100), or to an RMSD value of about 1.5, or to a B-value of about 60. For other types of models these values might vary, so you might need to experiment or use values that others have found useful.

After trimming low-confidence residues, you will usually be left with a model that has some complete parts of various sizes and some small pieces.

Splitting a trimmed model into domains

It can be helpful to group the pieces from your trimmed model into compact domains, or even to split some pieces into compact domains. The `process_predicted_model` tool allows you to choose a typical domain size, and if you want, a maximum number of domains, and then it will try to split your model into compact domains.

There are two methods available. One is based finding compact domains, the other is based on using the predicted alignment error matrix (AlphaFold2 only).

Finding domains from a low-resolution model representation

The method used is to calculate a low-resolution map based on the input model, then to identify large blobs corresponding to domains. All the residues in the structure are assigned to an initial domain. Then the residues are regrouped in order to have as few cases where small parts of the model are part of one domain but neighboring parts are part of another as possible.

When using this method, you can easily adjust the number of domains you get by adjusting the target domain size (in Å). You can also just restrict the number using the `maximum_domains` keyword (less good).

Finding domains using the predicted alignment error matrix

This method analyzes the predicted alignment error matrix (PAE) provided by AlphaFold2 and finds groupings of residues that have small mutual alignment error. This often corresponds to domains.

When using this method you can adjust the number of domains by changing the value of `pae_power` (the exponent applied to pae before using it in finding domains). You can also just restrict the number using the `maximum_domains` keyword (less good).

The pLDDT confidence measure

AlphaFold provides residue-level estimates of model accuracy in the form of predicted values of the LDDT (Local Difference Distance Test, Mariani et al., 2013). These predicted values are referred to as pLDDT values (predicted Local Difference Distance Test). The LDDT measure is a number from zero to one that reflects the similarity of CA positions in two structures.

The LDDT is not a simple measure of accuracy, but rather a composite that is based on the CA distances between all pairs of CA atoms in one model and how similar these inter-atomic distances are to those in the other model. It is a local test, where only inter-atomic distances that are less than 15 Å are considered.

The LDDT is a composite that is generated from the fraction of CA-CA distances that are very accurate (less than 0.5 Å off), the fraction that are quite accurate (less than 1 Å off), moderately accurate (less than 2 Å off), and somewhat accurate (less than 4 Å off). If all CA-CA distances are within 0.5 Å, the resulting score will be 1, if all are more than 4 Å off, the score is zero. Note that the LDDT is related to RMSD, but not in a simple way.

The pLDDT values (predicted Local Difference Distance Test) provided by AlphaFold are estimates of the LDDT values. The pLDDT values are essentially unbiased (just as likely to be too low as too high), and reasonably accurate. The correlation between pLDDT values and actual LDDT values calculated using AlphaFold models and models in the PDB is about 0.7 - 0.75, which means that the values of pLDDT you get are useful indicators of model quality but also that sometimes AlphaFold will have high confidence in an incorrect prediction or low confidence in a correct prediction.

Examples

Standard run of process_predicted_model:

Running process_predicted_model is easy. From the command-line you can type:

```
phenix.process_predicted_model my_model.pdb b_value_field_is=lddt
```

This will convert the B-value field in my_model.pdb based from LDDT to B-values, trim residues with LDDT less than 0.7, and write out a new model with individual chains (separate chain ID values) corresponding to compact domains.

What to do next after ProcessPredictedModel

If you have X-ray data, the next step after ProcessPredictedModel is usually molecular replacement with Phaser using your processed model as a search model, or automated molecular replacement and rebuilding with PredictAndBuild.

If you have cryo-EM data, the next step after ProcessPredictedModel is usually docking your model with EMPlacement or DockInMap , or automated docking and rebuilding with PredictAndBuild.

Note that ProcessPredictedModel normally is applied to a model that is not yet aligned with your density map for cryo-EM or for your crystal unit cell in crystallography. This means you normally must carry out some step for placing and orienting your ProcessPredictedModel model before carrying out refinement or any steps that involve comparing your model to data.

The program ProcessPredictedModel may be run as part of the larger PredictAndBuild tool, but it can also be run separately on predicted models obtained with PredictModel or from other sources.

Possible Problems

Specific limitations and problems:

ProcessPredictedModel is designed for processing single chains (a model with a single chain ID).

ProcessPredictedModel can be used for models with multiple chain IDs, but you cannot supply a reference structure or a PAE matrix if there are multiple chain IDs. In this case it will run each chain individually, then simply combine them to form the output processed model.

A new set of default parameters for some parameters are available for use with a pae matrix. The original values are:

```
minimum_domain_length = 10
minimum_sequential_residues = 5
pae_power = 1
pae_cutoff = 5
pae_graph_resolution = 0.5
```

Alternative parameters are available with the keyword "alt_pae_params=True" (or "Use PAE defaults for pae_power=2" in the GUI). These parameters may be helpful in cases where small domains are present. The alternative parameters are:

```
minimum_domain_length = 20
minimum_sequential_residues = 10
pae_power = 2
pae_cutoff = 4
pae_graph_resolution = 4
```

These parameters can also be set directly.

Literature

Highly accurate protein structure prediction with AlphaFold. J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S.A.A. Kohl, A.J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A.W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Nature 596, 583-589 (2021). IDDT: a local superposition-free score for comparing protein structures and models using distance difference tests. V. Mariani, M. Biasini, A. Barbato, and T.. Schwede. Bioinformatics 29, 2722-2728 (2013).

- Highly accurate protein structure prediction with AlphaFold. J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S.A.A. Kohl, A.J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A.W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Nature 596, 583-589 (2021).
- IDDT: a local superposition-free score for comparing protein structures and models using distance difference tests. V. Mariani, M. Biasini, A. Barbato, and T.. Schwede. Bioinformatics 29, 2722-2728 (2013).

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
 input_fileschain_id = None If specified, find domains in this chain only. NOTE: only one chain can be used for finding domains at a time.
 selection = None If specified, use only selected part of model
 pae_file = None Optional input json file with matrix of inter-residue estimated errors (pae file)
 distance_model_file = None Distance_model_file. A PDB or mmCIF file containing the model corresponding to the PAE matrix. Only needed if weight_by_ca_ca_distances is True and you want to specify a file other than your model file. (Default is to use the model file)
 model = None Input predicted model (e.g., AlphaFold model). Assumed to have LDDT values in B-value field (or RMSD values).
 output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
 processed_model_prefix = None Output file with processed models will begin with this prefix. If not specified, the input model file name will be used with the suffix _processed.
 remainder_seq_file_prefix = None Output file with sequences of deleted parts of model will begin with this
 prefixmark_atoms_to_keep_with_occ_one = False Mark atoms to keep with occupancy of 1 and those to remove with zero (default is to remove those that are not desired)
 maximum_output_b = 999. Limit output B values (so that they fit in old-style PDB format). Note that this limit is applied just before writing output model files, it does not affect anything else. Also if output model is trimmed normally, high-B value residues are already removed, so in most cases this keyword has no effect.
 remove_hydrogen = True Remove hydrogen atoms from model on input
 single_letter_chain_ids = False Write output files with all chain IDs as single characters. Default is to use original chain ID and to add digits (1-9) for domains.
 process_predicted_modelremove_low_confidence_residues = True Remove low-confidence residues (based on minimum plddt or maximum_rmsd, whichever is specified)
 continuous_chain = False When removing low-confidence residues, only trim from end
 split_model_by_compact_regions = True Split model into compact regions after removing low-confidence residues.
 maximum_domains = 3 Maximum domains to obtain. You can use this to merge the closest domains at the end of splitting the model. Make it bigger (and optionally make domain_size smaller) to get more domains. If model is processed in chunks, maximum_domains will apply to each chunk.
 domain_size = 15 Approximate size of domains to be found (A units). This is the resolution that will be used to make a domain map. If you are getting too many domains, try making domain_size bigger (maximum is 70 A).
 adjust_domain_size = True If more than maximum_domains are initially found, increase domain_size in increments of 5 A and take the value that gives the smallest number of domains, but at least maximum_domains.
 minimum_domain_length = None Minimum length of a domain to keep (reject at end if smaller). Default is 10 if no pae matrix or alt_pae_params=False and 20 for pae_matrix with alt_pae_params=True
 maximum_fraction_close = 0.3 Maximum fraction of CA in one domain close to one in another before merging them
 minimum_sequential_residues = None Minimum length of a short segment to keep (reject at end). Default is 5 if no pae matrix or alt_pae_params=False and 4 for pae_matrix with alt_pae_params=True
 minimum_remainder_sequence_length = 15 used to choose whether the sequence of a removed segment is written to the remainder sequence file.
 b_value_field_is = *plddt rmsd b_value The B-factor field in predicted models can be pLDDT (confidence, 0-1 or 0-100) or rmsd (A) or a B-factor
 input_plddt_is_fractional = None You can specify if the input plddt values (in B-factor field) are fractional (0-1) or not (0-100). By default if all values are between 0 and 1 it is fractional.

- job_title = None Job title in PHENIX GUI, not used on command line
- input_fileschain_id = None If specified, find domains in this chain only. NOTE: only one chain can be used for finding domains at a time.
 selection = None If specified, use only selected part of model
 pae_file = None Optional input json file with matrix of inter-residue estimated errors (pae file)
 distance_model_file = None Distance_model_file. A PDB or mmCIF file containing the model corresponding to the PAE matrix. Only needed if weight_by_ca_ca_distances is True and you want to specify a file other than your model file. (Default is to use the model file)
 model = None Input predicted model (e.g., AlphaFold model). Assumed to have LDDT values in B-value field (or RMSD values).

- `chain_id = None` If specified, find domains in this chain only. NOTE: only one chain can be used for finding domains at a time.
- `selection = None` If specified, use only selected part of model
- `pae_file = None` Optional input json file with matrix of inter-residue estimated errors (pae file)
- `distance_model_file = None` Distance_model_file. A PDB or mmCIF file containing the model corresponding to the PAE matrix. Only needed if `weight_by_ca_ca_distances` is True and you want to specify a file other than your model file. (Default is to use the model file)
- `model = None` Input predicted model (e.g., AlphaFold model). Assumed to have LDDT values in B-value field (or RMSD values).
- `output_filetarget_output_format = *None` pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
`processed_model_prefix = None` Output file with processed models will begin with this prefix. If not specified, the input model file name will be used with the suffix `_processed`.
`remainder_seq_file_prefix = None` Output file with sequences of deleted parts of model will begin with this prefix.
`mark_atoms_to_keep_with_occ_one = False` Mark atoms to keep with occupancy of 1 and those to remove with zero (default is to remove those that are not desired).
`maximum_output_b = 999`. Limit output B values (so that they fit in old-style PDB format). Note that this limit is applied just before writing output model files, it does not affect anything else. Also if output model is trimmed normally, high-B value residues are already removed, so in most cases this keyword has no effect.
`remove_hydrogen = True` Remove hydrogen atoms from model on input.
`single_letter_chain_ids = False` Write output files with all chain IDs as single characters. Default is to use original chain ID and to add digits (1-9) for domains.
- `target_output_format = *None` pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `processed_model_prefix = None` Output file with processed models will begin with this prefix. If not specified, the input model file name will be used with the suffix `_processed`.
- `remainder_seq_file_prefix = None` Output file with sequences of deleted parts of model will begin with this prefix
- `mark_atoms_to_keep_with_occ_one = False` Mark atoms to keep with occupancy of 1 and those to remove with zero (default is to remove those that are not desired)
- `maximum_output_b = 999`. Limit output B values (so that they fit in old-style PDB format). Note that this limit is applied just before writing output model files, it does not affect anything else. Also if output model is trimmed normally, high-B value residues are already removed, so in most cases this keyword has no effect.
- `remove_hydrogen = True` Remove hydrogen atoms from model on input
- `single_letter_chain_ids = False` Write output files with all chain IDs as single characters. Default is to use original chain ID and to add digits (1-9) for domains.
- `process_predicted_modelremove_low_confidence_residues = True` Remove low-confidence residues (based on minimum pLDDT or maximum_rmsd, whichever is specified).
`continuous_chain = False` When removing low-confidence residues, only trim from ends.
`split_model_by_compact_regions = True` Split model into compact regions after removing low-confidence residues.
`maximum_domains = 3` Maximum domains to obtain. You can use this to merge the closest domains at the end of splitting the model. Make it bigger (and optionally make domain_size smaller) to get more domains. If model is processed in

chunks, maximum_domains will apply to each chunk. domain_size = 15 Approximate size of domains to be found (A units). This is the resolution that will be used to make a domain map. If you are getting too many domains, try making domain_size bigger (maximum is 70 A). adjust_domain_size = True If more than maximum_domains are initially found, increase domain_size in increments of 5 A and take the value that gives the smallest number of domains, but at least maximum_domains. minimum_domain_length = None Minimum length of a domain to keep (reject at end if smaller). Default is 10 if no pae matrix or alt_pae_params=False and 20 for pae_matrix with alt_pae_params=True. maximum_fraction_close = 0.3 Maximum fraction of CA in one domain close to one in another before merging. minimum_sequential_residues = None Minimum length of a short segment to keep (reject at end). Default is 5 if no pae matrix or alt_pae_params=False and 4 for pae_matrix with alt_pae_params=True. minimum_remainder_sequence_length = 15 used to choose whether the sequence of a removed segment is written to the remainder sequence file. b_value_field_is = *plddt rmsd b_value The B-factor field in predicted models can be pLDDT (confidence, 0-1 or 0-100) or rmsd (A) or a B-factor. input_plddt_is_fractional = None You can specify if the input plddt values (in B-factor field) are fractional (0-1) or not (0-100). By default if all values are between 0 and 1 it is fractional. minimum_plddt = None If low-confidence residues are removed, the cutoff is defined by minimum_plddt or maximum_rmsd, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of input_plddt_is_fractional. maximum_rmsd = 1.5 If low-confidence residues are removed, the cutoff is defined by minimum_plddt or maximum_rmsd, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of input_plddt_is_fractional. default_maximum_rmsd = 1.5 Default value of maximum_rmsd, used if maximum_rmsd is not set. subtract_minimum_b = False If set, subtract the lowest B-value from all B-values just before writing out the final files. Does not affect the cutoff for removing low-confidence residues. pae_power = None If PAE matrix (predicted alignment error matrix) is supplied, each edge in the graph will be weighted proportional to $(1/\text{pae}^{\text{pae_power}})$. Use this to try and get the number of domains that you want (try 1, 0.5, 1.5, 2). Default is 1. alt_pae_params=False and 2 for alt_pae_params=True. pae_cutoff = None If PAE matrix (predicted alignment error matrix) is supplied, graph edges will only be created for residue pairs with $\text{pae} < \text{pae_cutoff}$. Default is 5. alt_pae_params=False and 4 for alt_pae_params=True. pae_graph_resolution = None If PAE matrix (predicted alignment error matrix) is supplied, pae_graph_resolution regulates how aggressively the clustering algorithm is. Smaller values lead to larger clusters. Value should be larger than zero, and values larger than 5 are unlikely to be useful. Default is 0.5. alt_pae_params=False and 4 for alt_pae_params=True. alt_pae_params = False If PAE matrix is supplied, use alternative set of defaults (minimum_domain_length=20 minimum_sequential_residues=10 pae_power=2 pae_cutoff=4 pae_graph_resolution=...

- remove_low_confidence_residues = True Remove low-confidence residues (based on minimum plddt or maximum_rmsd, whichever is specified)
- continuous_chain = False When removing low-confidence residues, only trim from ends
- split_model_by_compact_regions = True Split model into compact regions after removing low-confidence residues.
- maximum_domains = 3 Maximum domains to obtain. You can use this to merge the closest domains at the end of splitting the model. Make it bigger (and optionally make domain_size smaller) to get more domains. If model is processed in chunks, maximum_domains will apply to each chunk.
- domain_size = 15 Approximate size of domains to be found (A units). This is the resolution that will be used to make a domain map. If you are getting too many domains, try making domain_size bigger (maximum is 70 A).

- `adjust_domain_size = True` If more than `maximum_domains` are initially found, increase `domain_size` in increments of 5 Å and take the value that gives the smallest number of domains, but at least `maximum_domains`.
- `minimum_domain_length = None` Minimum length of a domain to keep (reject at end if smaller). Default is 10 if no pae matrix or `alt_pae_params=False` and 20 for pae_matrix with `alt_pae_params=True`
- `maximum_fraction_close = 0.3` Maximum fraction of CA in one domain close to one in another before merging them
- `minimum_sequential_residues = None` Minimum length of a short segment to keep (reject at end). Default is 5 if no pae matrix or `alt_pae_params=False` and 4 for pae_matrix with `alt_pae_params=True`
- `minimum_remainder_sequence_length = 15` used to choose whether the sequence of a removed segment is written to the remainder sequence file.
- `b_value_field_is = *plddt rmsd b_value` The B-factor field in predicted models can be pLDDT (confidence, 0-1 or 0-100) or rmsd (Å) or a B-factor
- `input_plddt_is_fractional = None` You can specify if the input plddt values (in B-factor field) are fractional (0-1) or not (0-100). By default if all values are between 0 and 1 it is fractional.
- `minimum_plddt = None` If low-confidence residues are removed, the cutoff is defined by `minimum_plddt` or `maximum_rmsd`, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of `input_plddt_is_fractional`.
- `maximum_rmsd = 1.5` If low-confidence residues are removed, the cutoff is defined by `minimum_plddt` or `maximum_rmsd`, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of `input_plddt_is_fractional`.
- `default_maximum_rmsd = 1.5` Default value of `maximum_rmsd`, used if `maximum_rmsd` is not set
- `subtract_minimum_b = False` If set, subtract the lowest B-value from all B-values just before writing out the final files. Does not affect the cutoff for removing low- confidence residues.
- `pae_power = None` If PAE matrix (predicted alignment error matrix) is supplied, each edge in the graph will be weighted proportional to $(1/\text{pae}^{\text{pae_power}})$. Use this to try and get the number of domains that you want (try 1, 0.5, 1.5, 2). Default is 1 `alt_pae_params=False` and 2 for `alt_pae_params=True`
- `pae_cutoff = None` If PAE matrix (predicted alignment error matrix) is supplied, graph edges will only be created for residue pairs with `pae < pae_cutoff`. Default is 5 `alt_pae_params=False` and 4 for `alt_pae_params=True`
- `pae_graph_resolution = None` If PAE matrix (predicted alignment error matrix) is supplied, `pae_graph_resolution` regulates how aggressively the clustering algorithm is. Smaller values lead to larger clusters. Value should be larger than zero, and values larger than 5 are unlikely to be useful, Default is 0.5 `alt_pae_params=False` and 4 for `alt_pae_params=True`
- `alt_pae_params = False` If PAE matrix is supplied, use alternative set of defaults (`minimum_domain_length=20` `minimum_sequential_residues=10` `pae_power=2` `pae_cutoff=4` `pae_graph_resolution=4`). Standard parameters (`alt_pae_params=False`) are: (`minimum_domain_length=10` `minimum_sequential_residues=5` `pae_power=1` `pae_cutoff=5` `pae_graph_resolution=0.5`).
- `weight_by_ca_ca_distance = False` Adjust the edge weighting for each residue pair according to the distance between CA residues. If this is True, then `distance_model` can be provided, otherwise supplied model will be used. See also `distance_power`

- `distance_power = 1` If `weight_by_ca_ca_distance` is `True`, then edge weights will be multiplied by $1/\text{distance}^{\text{distance_power}}$.
- `stop_if_no_residues_obtained = True` Raise Sorry and stop if processing yields no residues
- `keep_all_if_no_residues_obtained = False` Keep everything if processing yields no residues
- `vrms_from_rmsd_intercept = 0.25` Estimate of vrms (error in model) from pLDDT will be based on $\text{vrms_from_rmsd_intercept} + \text{vrms_from_rmsd_slope} * \text{pLDDT}$ where mean pLDDT of non-low_confidence_residues is used.
- `vrms_from_rmsd_slope = 1.0` Estimate of vrms (error in model) from pLDDT will be based on $\text{vrms_from_rmsd_intercept} + \text{vrms_from_rmsd_slope} * \text{pLDDT}$ where mean pLDDT of non-low_confidence_residues is used.
- `break_into_chunks_if_length_is = 1500` If a sequence is at least `break_into_chunks_if_length_is`, break it into chunks of length `chunk_size` with overlap of `overlap_size` for domain identification using `split_model_by_compact_regions` without a `pae_matrix`
- `chunk_size = 600` If a sequence is at least `break_into_chunks_if_length_is`, break it into chunks of length `chunk_size` with overlap of `overlap_size` for domain identification using `split_model_by_compact_regions` without a `pae_matrix`
- `overlap_size = 200` If a sequence is at least `break_into_chunks_if_length_is`, break it into chunks of length `chunk_size` with overlap of `overlap_size` for domain identification using `split_model_by_compact_regions` without a `pae_matrix`
- `controlwrite_files = True` Write output files
- `write_files = True` Write output files
- guiGUI-specific parameter required for output directory `output_dir = None`
- `output_dir = None`

Model searching and morphing using secondary structure matching (SSM)

[search_and_morph.html](#)

Author(s)

- `search_and_morph`: Tom Terwilliger

Purpose

Search for structures in the PDB similar to a supplied model using SSM matching, superimpose the matching structures, then optionally morph them to be similar to the target model and trim back to the parts that match.

Usage

How `search_and_morph` works:

`search_and_morph` uses SSM-based (secondary structure matching) to find structures in the PDB that are similar to a supplied model. The SSM matching is the same as in `phenix.superpose_and_morph`. The SSM matching starts with identifying secondary structure in the supplied model, indexing all pairs of secondary structure elements. These pairs of secondary structure elements are compared with indexed pairs from all structures in a curated subset of the PDB, leading to sets of matching pairs in the target and structures in the PDB.

The curated subset of the PDB consist of non-redundant (<90% identity), high-resolution structures (< 2 Å) that have very good geometry. The main-chain coordinates of these structures are supplied in a database as part of Phenix.

Normally the basic unit of a match is a set of three pairs of secondary structure elements (they can be overlapping) with the same spatial arrangement in the two structures. Such a match is called a triple in `search_and_morph`.

A matching triples yields a unique orientation of the moving model. Matching triples are grouped into sets in which all the triples yield nearly the same orientation. The PDB entries representing the largest groups are then tested for match to the target structure.

Optionally (if database is set to "pdb100"), sequence neighbors of these PDB entries are identified and added to the structures tested for matching the target structure

The structures to be examined are either loaded from the Phenix database or downloaded from the PDB. They are then superimposed on the target structure and (optionally) morphed and trimmed to match the target structure in the same way as in `phenix.superimpose_and_morph`. In this procedure the best-fitting superimposed structures are morphed by calculation of a real-space distortion field that changes over a typical distance of about 10 Å (set by the parameter `distortion_field_length`).

Finally (optionally) the moving models are trimmed to remove segments that are very different from the fixed model.

Examples

Standard run of search_and_morph:

Running search_and_morph is easy. From the command-line you can type:

```
phenix.search_and_morph model.pdb nproc=8
```

This will find structures in the PDB similar to model.pdb, superimpose them, morph them to match, and trim back to the matching region.

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
input_filesmodel = None Input PDB file to try and match
selection = None If specified, only include selected part of model
local_pdb_dir = None Local PDB mirror (something like /User/files/pdb_mirror/)
output_files superposed_model_prefix = None Output files with superposed models will begin with this prefix
search_and_morphmorph = True Morph moving model after superposition
trim = True Trim moving model after morphing
replace_structure_elements = True Replace each secondary structure element (helices/strands) with idealized version before finding matching segments in PDB. Includes replacing individual strands with helices if spacing from other secondary structure is suitable and density is compatible with a helix.
database = *pdb100 pdb2018_90 Database to search. pdb100 is unique set of PDB. pdb2018_90 is quality-filtered PDB at 90% identity
number_of_sequence_neighbors = None Number of sequence neighbors to search if using pdb100. Default is 10 for quick, 100 otherwise
max_models = 1 Number of models to return
allow_reverse_connectivity = False Allow match to reversed model
pdb_include_list = None List of PDB ID or pdb ID and chain_id to include (e.g., 2DY1 or 2DY1_A)
pdb_exclude_list = None List of PDB ID or pdb ID and chain_id to exclude (2DY1 or 2DY1_A)
exclude_similar_to_this_pdb_id = None Exclude all PDB chains with sequence identity > maximum_percent_identity_to_this_pdb_id to exclude_similar_to_this_pdb_id
maximum_percent_identity_to_this_pdb_id = 30 Maximum percent identity to supplied PDB chain to include. Exclude all PDB chains with sequence identity > maximum_percent_identity_to_this_pdb_id to exclude_similar_to_this_pdb_id
max_models_to_fetch = None Maximum number of models to fetch from PDB. Default is 10 for quick and 100 otherwise
starting_distance_cutoff = 2.5 Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.
shift_field_distance = None Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 A.
controlnproc = 1 Number of processors to use. If None, uses all available
quick = False Run quickly
guiGUI-specific parameter required for output directory
output_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesmodel = None Input PDB file to try and match
selection = None If specified, only include selected part of model
local_pdb_dir = None Local PDB mirror (something like /User/files/pdb_mirror/)
- model = None Input PDB file to try and match

- selection = None If specified, only include selected part of model
- local_pdb_dir = None Local PDB mirror (something like /User/files/pdb_mirror/)
- output_filesuperposed_model_prefix = None Output files with superposed models will begin with this prefix
- superposed_model_prefix = None Output files with superposed models will begin with this prefix
- search_and_morphmorph = True Morph moving model after superpositiontrim = True Trim moving model after morphingreplace_structure_elements = True Replace each secondary structure element (helices/strands) with idealized version before finding matching segments in PDB. Includes replacing individual strands with helices if spacing from other secondary structure is suitable and density is compatible with a helix.database = *pdb100 pdb2018_90 Database to search. pdb100 is unique set of PDB. pdb2018_90 is quality-filtered PDB at 90% identitynumber_of_sequence_neighbors = None Number of sequence neighbors to search if using pdb100. Default is 10 for quick, 100 otherwise
- max_models = 1 Number of models to returnallow_reverse_connectivity = False Allow match to reversed modelpdb_include_list = None List of PDB ID or pdb ID and chain_id to include (e.g., 2DY1 or 2DY1_A)pdb_exclude_list = None List of PDB ID or pdb ID and chain_id to exclude (2DY1 or 2DY1_A)exclude_similar_to_this_pdb_id = None Exclude all PDB chains with sequence identity > maximum_percent_identity_to_this_pdb_id to exclude_similar_to_this_pdb_idmaximum_percent_identity_to_this_pdb_id = 30 Maximum percent identity to supplied PDB chain to include. Exclude all PDB chains with sequence identity > maximum_percent_identity_to_this_pdb_id to exclude_similar_to_this_pdb_idmax_models_to_fetch = None Maximum number of models to fetch from PDB. Default is 10 for quick and 100 otherwise
- starting_distance_cutoff = 2.5 Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.shift_field_distance = None Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 A.
- morph = True Morph moving model after superposition
- trim = True Trim moving model after morphing
- replace_structure_elements = True Replace each secondary structure element (helices/strands) with idealized version before finding matching segments in PDB. Includes replacing individual strands with helices if spacing from other secondary structure is suitable and density is compatible with a helix.
- database = *pdb100 pdb2018_90 Database to search. pdb100 is unique set of PDB. pdb2018_90 is quality-filtered PDB at 90% identity
- number_of_sequence_neighbors = None Number of sequence neighbors to search if using pdb100. Default is 10 for quick, 100 otherwise
- max_models = 1 Number of models to return
- allow_reverse_connectivity = False Allow match to reversed model
- pdb_include_list = None List of PDB ID or pdb ID and chain_id to include (e.g., 2DY1 or 2DY1_A)
- pdb_exclude_list = None List of PDB ID or pdb ID and chain_id to exclude (2DY1 or 2DY1_A)
- exclude_similar_to_this_pdb_id = None Exclude all PDB chains with sequence identity > maximum_percent_identity_to_this_pdb_id to exclude_similar_to_this_pdb_id
- maximum_percent_identity_to_this_pdb_id = 30 Maximum percent identity to supplied PDB chain to include. Exclude all PDB chains with sequence identity > maximum_percent_identity_to_this_pdb_id to exclude_similar_to_this_pdb_id

- `max_models_to_fetch = None` Maximum number of models to fetch from PDB. Default is 10 for quick and 100 otherwise
- `starting_distance_cutoff = 2.5` Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.
- `shift_field_distance = None` Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 Å.
- `controlnproc = 1` Number of processors to use. If None, uses all available
- `quick = False` Run quickly
- `nproc = 1` Number of processors to use. If None, uses all available
- `quick = False` Run quickly
- `gui` GUI-specific parameter required for output directory
- `output_dir = None`
- `output_dir = None`

Model superposition and morphing using secondary structure matching (SSM) (deprecated, use `superpose_models`)

`superpose_and_morph.html`

Author(s)

- `superpose_and_morph`: Tom Terwilliger

Purpose

Superpose moving model on fixed model, then optionally morph the moving model to be similar to the fixed model and trim it back to the parts that match.

Usage

How `superpose_and_morph` works:

`superpose_and_morph` uses a combination of sequence-based and SSM-based (secondary structure matching) to superimpose two models. The sequence-based matching is the same as in `phenix.superpose_pdbs`. The SSM matching is the same as in `phenix.search_and_morph`. The SSM matching consists of identifying secondary structure in the two models, indexing all pairs of secondary structure elements, and finding sets of matching pairs in the two structures. Normally the basic unit of a match is a set of three pairs of secondary structure elements (they can be overlapping) with the same spatial arrangement in the two structures. Such a match is called a triple in `superpose_and_morph`.

A matching triples yields a unique orientation of the moving model. Matching triples are grouped into sets in which all the triples yield nearly the same orientation. Then the transformations suggested by the largest groups are each tested by using them to superimpose the moving model on the fixed model and noting the number of residues that superimpose.

The best-fitting superimposed moving models are then (optionally) morphed by calculation of a real-space distortion field that changes over a typical distance of about 10 Å (set by the parameter `distortion_field_length`).

Finally (optionally) the moving models are trimmed to remove segments that are very different from the fixed model.

Options

SSM match to map. You can supply a map and specify `ssm_match_to_map=True`. The tool will search for helices and strands in the map and generate a target `helices_strands` model. Then a brute force SSM matching of your moving model to the `helices_strands` model is carried out and matches are scored with map-model correlation. This option can also be carried out with the `dock_in_map` tool. Note this option does not support trimming or morphing. Brute force SSM. You can supply a target model that has helices and strands. Helices and strands in your moving model will be matched to those in the target model to superpose it. You can also supply a `map_file` and specify `score_superpose_brute_force_with_cc` if you wish. Note this option does not support trimming or morphing.

SSM match to map. You can supply a map and specify `ssm_match_to_map=True`. The tool will search for helices and strands in the map and generate a target `helices_strands` model. Then a brute force SSM matching of your moving model to the `helices_strands` model is carried out and matches are scored with map-model correlation. This option can also be carried out with the `dock_in_map` tool. Note this option does not support trimming or morphing.

Brute force SSM. You can supply a target model that has helices and strands. Helices and strands in your moving model will be matched to those in the target model to superpose it. You can also supply a `map_file` and specify `score_superpose_brute_force_with_cc` if you wish. Note this option does not support trimming or morphing.

Examples

Standard run of `superpose_and_morph`:

Running `superpose_and_morph` is easy. From the command-line you can type:

```
phenix.superpose_and_morph fixed_model=fixed_model.pdb \
moving_model=moving_model.pdb
```

This will superimpose `moving_model.pdb` on `fixed_model.pdb`, morph it (gradual adjustment of atomic positions with location along the chain and in space), and trim `moving_model` back to the region where they are similar.

To superpose a model on a map you can say:

```
phenix.superpose_and_morph map_file=map.ccp4 moving_model=moving_model.pdb
```

This will find helices and strands in `map.ccp4` and superpose `moving_model.pdb` on those secondary structure elements.

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

`job_title` = None Job title in PHENIX GUI, not used on command line
`input_files`
`fixed_model` = None Input fixed PDB file
`fixed_selection` = None If specified, only include selected part of fixed model
`moving_model` = None Input moving PDB file. Model to superpose on fixed_model
`moving_selection` = None If specified, only include selected part of moving model
`map_file` = None File with CCP4-style map for brute_force superposition
`only_include_first_chain` = False Only use first chain in each input model
`output_files`
`helices_strands_model` = 'helices_strands.pdb' Helices-strands model
`superposed_model` = None Output file with superposed moving model
`crystal_info`
`resolution` = None Resolution. Will be truncated to `minimum_working_resolution` if finer than that
`minimum_working_resolution` = 3 Working High-resolution limit. Resolution will be truncated to this value
`wrapping` = False Map_file is a crystal
`superpose_and_morph`
`superpose` = True Superpose moving model on fixed model
`morph` = True Morph

moving model after superpositiontrim = True Trim moving model after morphingssm_match_to_map = False
 SSM match to helices/strands from mapsuperpose_brute_force = False Use brute force method to
 superposescore_superpose_brute_force_with_cc = None Use cc scoring in brute force method (requires map
 file)ok_brute_force_cc = 0.25 OK value for brute force cc (stop looking if achieved)shift_field_distance = None
 Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances
 morphing changes model. Default is 10 Å.good_enough_fraction = 0.9 Good enough fraction. If this much of
 model is found with sequence-based search skip ssmstarting_distance_cutoff = 2.5 Starting distance cutoff.
 Try to keep atoms within this distance, but increase if too few are found.include_ssm = True Include ssm
 matchinginclude_lsq = True Include lsq matchingtry_lsq_first = True Try lsq matching first if lsq and ssm are
 both includedtry_with_fragments_if_poor_fit = True Try lsq matching with fragments if no fit
 obtainedinclude_swapped_match = None Try matching fixed to moving if initial match
 failsallow_reverse_connectivity = None Allow match to reversed modeldirect_morph = False Directly morph
 moving model onto fixed model. Requires chain ID and residue numbering to be identical, but fixed_model
 can have missing residues. Does not allow trimming.direct_morph_method = along_chain Choice of method
 for direct morph: along_chain or in_spacedirect_morph_smoothing_window = 11 Smoothing window for direct
 morphidentify_register_shifts = False Identify positions in chains where the register between fixed and moving
 models changescontrolnproc = 1 Number of processors to use. If None, uses all availableguiGUI-specific
 parameter required for output directoryoutput_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesfixed_model = None Input fixed PDB filefixed_selection = None If specified, only include
selected part of fixed modelmoving_model = None Input moving PDB file. Model to superpose on
fixed_modelmoving_selection = None If specified, only include selected part of moving modelmap_file =
None File with CCP4-style map for brute_force superposition onlyonly_include_first_chain = False Only
use first chain in each input model
- fixed_model = None Input fixed PDB file
- fixed_selection = None If specified, only include selected part of fixed model
- moving_model = None Input moving PDB file. Model to superpose on fixed_model
- moving_selection = None If specified, only include selected part of moving model
- map_file = None File with CCP4-style map for brute_force superposition only
- only_include_first_chain = False Only use first chain in each input model
- output_files helices_strands_model = 'helices_strands.pdb' Helices-strands modelsuperposed_model =
None Output file with superposed moving model
- helices_strands_model = 'helices_strands.pdb' Helices-strands model
- superposed_model = None Output file with superposed moving model
- crystal_inforesolution = None Resolution. Will be truncated to minimum_working_resolution if finer than
thatminimum_working_resolution = 3 Working High-resolution limit. Resolution will be truncated to this
valuewrapping = False Map_file is a crystal
- resolution = None Resolution. Will be truncated to minimum_working_resolution if finer than that
- minimum_working_resolution = 3 Working High-resolution limit. Resolution will be truncated to this
value
- wrapping = False Map_file is a crystal
- superpose_and_morphsuperpose = True Superpose moving model on fixed modelmorph = True Morph
moving model after superpositiontrim = True Trim moving model after morphingssm_match_to_map =

False SSM match to helices/strands from map
 superpose_brute_force = False Use brute force method to superpose
 score_superpose_brute_force_with_cc = None Use cc scoring in brute force method (requires map file)
 ok_brute_force_cc = 0.25 OK value for brute force cc (stop looking if achieved)
 shift_field_distance = None Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 Å.
 good_enough_fraction = 0.9 Good enough fraction. If this much of model is found with sequence-based search skip ssm
 starting_distance_cutoff = 2.5 Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.
 include_ssm = True Include ssm matching
 include_lsq = True Include lsq matching
 try_lsq_first = True Try lsq matching first if lsq and ssm are both included
 try_with_fragments_if_poor_fit = True Try lsq matching with fragments if no fit obtained
 include_swapped_match = None Try matching fixed to moving if initial match fails
 allow_reverse_connectivity = None Allow match to reversed model
 direct_morph = False Directly morph moving model onto fixed model. Requires chain ID and residue numbering to be identical, but fixed_model can have missing residues. Does not allow trimming.
 direct_morph_method = along_chain Choice of method for direct morph: along_chain or in_space
 direct_morph_smoothing_window = 11 Smoothing window for direct morph
 identify_register_shifts = False Identify positions in chains where the register between fixed and moving models changes

- superpose = True Superpose moving model on fixed model
- morph = True Morph moving model after superposition
- trim = True Trim moving model after morphing
- ssm_match_to_map = False SSM match to helices/strands from map
- superpose_brute_force = False Use brute force method to superpose
- score_superpose_brute_force_with_cc = None Use cc scoring in brute force method (requires map file)
- ok_brute_force_cc = 0.25 OK value for brute force cc (stop looking if achieved)
- shift_field_distance = None Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 Å.
- good_enough_fraction = 0.9 Good enough fraction. If this much of model is found with sequence-based search skip ssm
- starting_distance_cutoff = 2.5 Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.
- include_ssm = True Include ssm matching
- include_lsq = True Include lsq matching
- try_lsq_first = True Try lsq matching first if lsq and ssm are both included
- try_with_fragments_if_poor_fit = True Try lsq matching with fragments if no fit obtained
- include_swapped_match = None Try matching fixed to moving if initial match fails
- allow_reverse_connectivity = None Allow match to reversed model
- direct_morph = False Directly morph moving model onto fixed model. Requires chain ID and residue numbering to be identical, but fixed_model can have missing residues. Does not allow trimming.
- direct_morph_method = along_chain Choice of method for direct morph: along_chain or in_space
- direct_morph_smoothing_window = 11 Smoothing window for direct morph
- identify_register_shifts = False Identify positions in chains where the register between fixed and moving models changes

- `controlnproc = 1` Number of processors to use. If `None`, uses all available
- `nproc = 1` Number of processors to use. If `None`, uses all available
- guiGUI-specific parameter required for output directory `output_dir = None`
- `output_dir = None`

phenix.superpose_maps

superpose_maps.html

Authors

Pavel Afonine, Nat Echols

- Pavel Afonine, Nat Echols

Guidelines for use

phenix.superpose_maps is a utility for transforming two maps to the same frame of reference, allowing direct comparison of molecular features in electron density. The name is slightly misleading, because the program actually performs a superposition of PDB files, then reorients the associated maps to follow them. This is usually done for publication purposes, to compare electron density for the same feature(s) in different crystal forms. The examples shown here are structures of the protein kinase PKA, one in space group P4(2) (PDB ID 1syk), one in space group P2(1)2(1)2(1) (PDB ID 3fhi). (These are included in the PHENIX distribution as part of the "pka-compare" example.)

We recommend running this program from the GUI, and this document covers the GUI version; however, the command-line program should work equally well, and parameters are listed below.

The program can be started from the main GUI by clicking "Superpose maps" in the "Maps" category. A PDB file and an MTZ file containing map coefficients are required for each structure. Map column labels will be extracted automatically and added to the drop-down menus.

The program will create a new directory named "SuperposeMaps_X", where X is the job ID (this applies to the GUI version only). Output is in either CCP4 or XPLOR map format (we recommend CCP4 maps because of the reduced file size). These can be automatically loaded into Coot or PyMOL once the program is finished running.

Caveats and known limitations

Output file names are determined automatically; this will be fixed in a future version. Only one chain is used in the molecular superposition at present. The program will try to guess the appropriate pair of chains to use, but you can specify alternate chains if you prefer. You may only transform one pair of maps at a time; this limitation will be removed in future versions. In most cases, the same input parameters (except MTZ column labels) should always result in the same frame of reference for the output maps, so the program may be run serially if multiple sets of maps are involved. Both input models will be transformed to ensure that all coordinates are positive (placing the new map in a simple P1 box). For the first model, only a translation will be involved. There is currently no simple way to recover the initial frames of reference for the input models; however, the structure comparison GUI does include this feature (allowing editing of transformed models in Coot, followed by refinement in the original orientations), although it is limited to a few pre-defined map types. Because maps are sampled along a simple grid, reorientation of the map requires re-sampling for the new grid. This requires interpolation of the new grid values, and some accuracy will be lost in the process (partly depending on the orientation of the new grid relative to the original). We have not observed artifacts in the transformed electron density, but the grid point values are not precise, and further interpolation is not advised. All symmetry in the original structures is lost. Although the regions of electron density directly adjacent to the molecules will be preserved, there will be a noticeable break where the map file ends.

- Output file names are determined automatically; this will be fixed in a future version.
- Only one chain is used in the molecular superposition at present. The program will try to guess the appropriate pair of chains to use, but you can specify alternate chains if you prefer.
- You may only transform one pair of maps at a time; this limitation will be removed in future versions. In most cases, the same input parameters (except MTZ column labels) should always result in the same frame of reference for the output maps, so the program may be run serially if multiple sets of maps are involved.
- Both input models will be transformed to ensure that all coordinates are positive (placing the new map in a simple P1 box). For the first model, only a translation will be involved.
- There is currently no simple way to recover the initial frames of reference for the input models; however, the structure comparison GUI does include this feature (allowing editing of transformed models in Coot, followed by refinement in the original orientations), although it is limited to a few pre-defined map types.
- Because maps are sampled along a simple grid, reorientation of the map requires re-sampling for the new grid. This requires interpolation of the new grid values, and some accuracy will be lost in the process (partly depending on the orientation of the new grid relative to the original). We have not observed artifacts in the transformed electron density, but the grid point values are not precise, and further interpolation is not advised.
- All symmetry in the original structures is lost. Although the regions of electron density directly adjacent to the molecules will be preserved, there will be a noticeable break where the map file ends.

References

Wu J, Yang J, Kannan N, Madhusudan, Xuong NH, Ten Eyck LF, Taylor SS. Crystal structure of the E230Q mutant of cAMP-dependent protein kinase reveals an unexpected apoenzyme conformation and an extended N-terminal A helix. *Protein Sci.* 2005 14:2871-9. PubMed PMID: 16253959; PDB ID 1syk
 Kim C, Xuong NH, Taylor SS. Crystal structure of a complex between the catalytic and regulatory (R1alpha) subunits of PKA. *Science* 2005 307:690-6. PubMed PMID: 15692043; PDB ID 3fhi

- Wu J, Yang J, Kannan N, Madhusudan, Xuong NH, Ten Eyck LF, Taylor SS. Crystal structure of the E230Q mutant of cAMP-dependent protein kinase reveals an unexpected apoenzyme conformation and an extended N-terminal A helix. *Protein Sci.* 2005 14:2871-9. PubMed PMID: 16253959; PDB ID 1syk
- Kim C, Xuong NH, Taylor SS. Crystal structure of a complex between the catalytic and regulatory (R1alpha) subunits of PKA. *Science* 2005 307:690-6. PubMed PMID: 15692043; PDB ID 3fhi

List of all available keywords

verbose = False dry_run = False superpose_mapsinput_filesmap_1 = None Map coefficients in MTZ
 formatpdb_1 = None PDB file associated with map_1map_2 = None Map coefficients in MTZ formatpdb_2 =
 None labels_1 = None MTZ column labels for maplabels_2 = None MTZ column labels for mapd_min_1 =
 None High resolution cutoff for map_1d_min_2 = None High resolution cutoff for map_2outputoutput_dir =
 None region = *cell selection Region to coverresolution_factor = 0.3333 Map grid spacing (multiplied times
 resolution)map_format = *ccp4 xplor buffer = 10.0 Padding around moleculejob_title = None Job title in
 PHENIX GUI, not used on command linemiscis_difference_map = False superpose_pdbsselection_fixed =
 None Selection of the target atoms to fit to (optional)selection_moving = None Selection of the atoms that will
 be fit to selection_fixed (optional)selection_default_moving = pepnames and (name ca or name n or name c)
 and '(altloc " " or altloc "A")' Select protein atoms that will be used in superposition after sequence
 alignmentselection_default_fixed = pepnames and (name ca or name n or name c) and '(altloc " " or altloc

"A")' Select protein atoms that will be used in superposition after sequence alignment
 reciprocal_matching = False try both orientations for homodimers
 alignmentSet of parameters for sequence alignment. Defaults are good for most of cases
 alignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1
 similarity_matrix = blosum50 dayhoff *identity

- verbose = False
- dry_run = False
- superpose_maps
 input_filesmap_1 = None Map coefficients in MTZ format
 pdb_1 = None PDB file associated with map_1
 map_2 = None Map coefficients in MTZ format
 pdb_2 = None labels_1 = None MTZ column labels for map
 labels_2 = None MTZ column labels for map
 d_min_1 = None High resolution cutoff for map_1
 d_min_2 = None High resolution cutoff for map_2
 output_dir = None region = *cell selection Region to cover
 resolution_factor = 0.3333 Map grid spacing (multiplied times resolution)
 map_format = *ccp4 xplor buffer = 10.0 Padding around molecule
 job_title = None Job title in PHENIX GUI, not used on command line
- selection
 Region to cover
 resolution_factor = 0.3333 Map grid spacing (multiplied times resolution)
 map_format = *ccp4 xplor buffer = 10.0 Padding around molecule
 job_title = None Job title in PHENIX GUI, not used on command line
- misc
 difference_map = False superpose_pdb
 selection_fixed = None Selection of the target atoms to fit to (optional)
 selection_moving = None Selection of the atoms that will be fit to
 selection_fixed (optional)
 selection_default_moving = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")'
 ' Select protein atoms that will be used in superposition after sequence alignment
 selection_default_fixed = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")'
 ' Select protein atoms that will be used in superposition after sequence alignment
 reciprocal_matching = False try both orientations for homodimers
 alignmentSet of parameters for sequence alignment. Defaults are good for most of cases
 alignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1 similarity_matrix = blosum50 dayhoff *identity
- input_files
 map_1 = None Map coefficients in MTZ format
 pdb_1 = None PDB file associated with map_1
 map_2 = None Map coefficients in MTZ format
 pdb_2 = None labels_1 = None MTZ column labels for map
 labels_2 = None MTZ column labels for map
 d_min_1 = None High resolution cutoff for map_1
 d_min_2 = None High resolution cutoff for map_2
- map_1 = None Map coefficients in MTZ format
- pdb_1 = None PDB file associated with map_1
- map_2 = None Map coefficients in MTZ format
- pdb_2 = None
- labels_1 = None MTZ column labels for map
- labels_2 = None MTZ column labels for map
- d_min_1 = None High resolution cutoff for map_1
- d_min_2 = None High resolution cutoff for map_2
- output_dir = None region = *cell selection Region to cover
 resolution_factor = 0.3333 Map grid spacing (multiplied times resolution)
 map_format = *ccp4 xplor buffer = 10.0 Padding around molecule
 job_title = None Job title in PHENIX GUI, not used on command line
- region = *cell selection Region to cover
- resolution_factor = 0.3333 Map grid spacing (multiplied times resolution)
- map_format = *ccp4 xplor
- buffer = 10.0 Padding around molecule
- job_title = None Job title in PHENIX GUI, not used on command line

- `miscis_difference_map = False`
- `is_difference_map = False`
- `superpose_pdbselection_fixed = None` Selection of the target atoms to fit to (optional)
`selection_moving = None` Selection of the atoms that will be fit to `selection_fixed` (optional)
`selection_default_moving = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")'` Select protein atoms that will be used in superposition after sequence alignment
`selection_default_fixed = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")'` Select protein atoms that will be used in superposition after sequence alignment
`reciprocal_matching = False` try both orientations for homodimers
`alignmentSet` of parameters for sequence alignment. Defaults are good for most of cases
`alignment_style = local *global`
`gap_opening_penalty = 1`
`gap_extension_penalty = 1`
`similarity_matrix = blosum50 dayhoff *identity`
- `selection_fixed = None` Selection of the target atoms to fit to (optional)
- `selection_moving = None` Selection of the atoms that will be fit to `selection_fixed` (optional)
- `selection_default_moving = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")'`
Select protein atoms that will be used in superposition after sequence alignment
- `selection_default_fixed = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")'`
Select protein atoms that will be used in superposition after sequence alignment
- `reciprocal_matching = False` try both orientations for homodimers
- `alignmentSet` of parameters for sequence alignment. Defaults are good for most of cases
`alignment_style = local *global`
`gap_opening_penalty = 1`
`gap_extension_penalty = 1`
`similarity_matrix = blosum50 dayhoff *identity`
- `alignment_style = local *global`
- `gap_opening_penalty = 1`
- `gap_extension_penalty = 1`
- `similarity_matrix = blosum50 dayhoff *identity`

Model superposition with optional morphing and trimming

superpose_models.html

Author(s)

- superpose_models: Peter Zwart, Pavel Afonine, Ralf W. Grosse-Kunstleve, Tom Terwilliger

superpose_models: Peter Zwart, Pavel Afonine, Ralf W. Grosse-Kunstleve,
Tom Terwilliger

Purpose

Superpose moving model on fixed model. Optionally morph the moving model to be similar to the fixed model and trim it back to the parts that match.

Usage

How superpose_models works:

The superpose_models tool can use either sequence-based alignment with least-squares fitting or SSM-based (secondary-structure-matching) superposition.

LSQ fitting (sequence-based alignment and least-squares fitting)

The LSQ fitting method consists of least-squares superposition of two selected parts from two pdb files. If no selections is provided for fixed and moving models the whole content of both input PDB files is used for superposition. If the number of atoms in fixed and moving models is different and the models contain amino-acid residues then the sequence alignment is performed and the matching residues (CA atoms by default, can be changed by the user) are used for superposition. Note that selected (and/or matching) atoms are the atoms used to find the superposition operators while these operators are applied to the whole moving structure.

SSM matching

SSM matching in phenix.superpose_models consists of identifying secondary structure in the two models, indexing all pairs of secondary structure elements, and finding sets of matching pairs in the two structures. Normally the basic unit of a match is a set of three pairs of secondary structure elements (they can be overlapping) with the same spatial arrangement in the two structures. Such a match is called a triple in superpose_models.

A matching triples yields a unique orientation of the moving model. Matching triples are grouped into sets in which all the triples yield nearly the same orientation. Then the transformations suggested by the largest groups are each tested by using them to superimpose the moving model on the fixed model and noting the number of residues that superimpose.

Options

LSQ fitting. This is the default superposition method. SSM matching. This is an alternative to LSQ fitting. It can be useful if your models do not have similar sequences Morphing. The best-fitting superimposed moving

models can be morphed by calculation of a real-space distortion field that changes over a typical distance of about 10 Å (set by the parameter `distortion_field_length`). This is a way to superpose one model on another and then make them as similar as possible by applying a smooth distortion to the moving model. Trimming. The superposed (and optionally morphed) moving model can be trimmed to remove segments that are very different from the fixed model. SSM match to map. You can supply a map and specify `ssm_match_to_map=True`. The tool will search for helices and strands in the map and generate a target `helices_strands` model. Then a brute force SSM matching of your moving model to the `helices_strands` model is carried out and matches are scored with map-model correlation. This option can also be carried out with the `dock_in_map` tool. Note this option does not support trimming or morphing. Brute force SSM. You can supply a target model that has helices and strands. Helices and strands in your moving model will be matched to those in the target model to superpose it. You can also supply a `map_file` and specify `score_superpose_brute_force_with_cc` if you wish. Note this option does not support trimming or morphing.

LSQ fitting. This is the default superposition method.

SSM matching. This is an alternative to LSQ fitting. It can be useful if your models do not have similar sequences

Morphing. The best-fitting superimposed moving models can be morphed by calculation of a real-space distortion field that changes over a typical distance of about 10 Å (set by the parameter `distortion_field_length`). This is a way to superpose one model on another and then make them as similar as possible by applying a smooth distortion to the moving model

Trimming. The superposed (and optionally morphed) moving model can be trimmed to remove segments that are very different from the fixed model.

SSM match to map. You can supply a map and specify `ssm_match_to_map=True`. The tool will search for helices and strands in the map and generate a target `helices_strands` model. Then a brute force SSM matching of your moving model to the `helices_strands` model is carried out and matches are scored with map-model correlation. This option can also be carried out with the `dock_in_map` tool. Note this option does not support trimming or morphing.

Brute force SSM. You can supply a target model that has helices and strands. Helices and strands in your moving model will be matched to those in the target model to superpose it. You can also supply a `map_file` and specify `score_superpose_brute_force_with_cc` if you wish. Note this option does not support trimming or morphing.

Examples

Standard run of `superpose_models`:

Running `superpose_models` is easy. From the command-line you can type:

```
phenix.superpose_models fixed_model=fixed_model.pdb \
moving_model=moving_model.pdb morph=true trim=true
```

This will superimpose `moving_model.pdb` on `fixed_model.pdb`, morph it (gradual adjustment of atomic positions with location along the chain and in space), and trim `moving_model` back to the region where they are similar.

To superpose a model on a map you can say:

```
phenix.superpose_models map_file=map.ccp4 moving_model=moving_model.pdb
```

This will find helices and strands in map.ccp4 and superpose moving_model.pdb on those secondary structure elements.

Possible Problems

Specific limitations and problems:

- When using LSQ fitting a different number of atoms in selection_fixed and selection_moving can lead to failure if sequence alignment is not successful.
- More than one model in one PDB file (separated with MODEL-ENDMDL) is not supported.

Literature

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
input_filesfixed_model = None Input fixed PDB file
fixed_selection = None If specified, only include selected part of fixed model
moving_model = None Input moving PDB file. Model to superpose on
fixed_modelmoving_selection = None If specified, only include selected part of moving model
other_model_to_move = None If specified, transform this model with the same parameters as the moving model
map_file = None File with CCP4-style map for brute_force superposition
only_include_first_chain = False Only use first chain in each input
modeloutput_files
helices_strands_model = 'helices_strands.pdb' Helices-strands model
superposed_model = None Output file with superposed moving model
transformed_other_model = None Output file with moved other_model
crystal_inforesolution = None Resolution. Will be truncated to minimum_working_resolution if finer than that
minimum_working_resolution = 3 Working High-resolution limit. Resolution will be truncated to this value
wrapping = False Map_file is a crystal
superpose_modelssuperpose = True Superpose moving model on fixed model
morph = True Morph moving model after superposition
trim = True Trim moving model after morphing
ssm_match_to_map = False SSM match to helices/strands from map
superpose_brute_force = False Use brute force method to superpose
score_superpose_brute_force_with_cc = None Use cc scoring in brute force method (requires map file)
ok_brute_force_cc = 0.25 OK value for brute force cc (stop looking if achieved)
shift_field_distance = None Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 Å.
good_enough_fraction = 0.9 Good enough fraction. If this much of model is found with sequence-based search skip ssm
starting_distance_cutoff = 2.5 Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.
include_ssm = True Include SSM matching (superpose based on secondary structure)
include_lsq = True Include least-squares matching (minimizing RMSD)
try_lsq_first = True Try lsq matching first if lsq and ssm are both included
try_with_fragments_if_poor_fit = True Try lsq matching with fragments if no fit obtained
include_swapped_match = None Try matching fixed to moving if initial match fails
allow_reverse_connectivity = None Allow match to reversed model
direct_morph = False Directly morph moving model onto fixed model. Requires chain ID and residue numbering to be identical, but fixed_model can have missing residues. Does not allow trimming.
direct_morph_method = along_chain Choice of method for direct morph: along_chain or in_space
direct_morph_smoothing_window = 11 Smoothing window for direct morph
identify_register_shifts = False Identify positions in chains where the register between fixed and moving models changes
alignmentSet of parameters for sequence alignment. Defaults are good for most of cases
alignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1 similarity_matrix =

blosum50 dayhoff *identity controlnproc = 1 Number of processors to use. If None, uses all availableguiGUI-specific parameter required for output directoryoutput_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesfixed_model = None Input fixed PDB filefixed_selection = None If specified, only include selected part of fixed modelmoving_model = None Input moving PDB file. Model to superpose on fixed_modelmoving_selection = None If specified, only include selected part of moving modelother_model_to_move = None If specified, transform this model with the same parameters as the moving_modelmap_file = None File with CCP4-style map for brute_force superposition onlyonly_include_first_chain = False Only use first chain in each input model
- fixed_model = None Input fixed PDB file
- fixed_selection = None If specified, only include selected part of fixed model
- moving_model = None Input moving PDB file. Model to superpose on fixed_model
- moving_selection = None If specified, only include selected part of moving model
- other_model_to_move = None If specified, transform this model with the same parameters as the moving_model
- map_file = None File with CCP4-style map for brute_force superposition only
- only_include_first_chain = False Only use first chain in each input model
- output_files helices_strands_model = 'helices_strands.pdb' Helices-strands modelsuperposed_model = None Output file with superposed moving modeltransformed_other_model = None Output file with moved other_model
- helices_strands_model = 'helices_strands.pdb' Helices-strands model
- superposed_model = None Output file with superposed moving model
- transformed_other_model = None Output file with moved other_model
- crystal_inforesolution = None Resolution. Will be truncated to minimum_working_resolution if finer than thatminimum_working_resolution = 3 Working High-resolution limit. Resolution will be truncated to this valuewrapping = False Map_file is a crystal
- resolution = None Resolution. Will be truncated to minimum_working_resolution if finer than that
- minimum_working_resolution = 3 Working High-resolution limit. Resolution will be truncated to this value
- wrapping = False Map_file is a crystal
- superpose_modelssuperpose = True Superpose moving model on fixed modelmorph = True Morph moving model after superpositiontrim = True Trim moving model after morphingssm_match_to_map = False SSM match to helices/strands from mapsuperpose_brute_force = False Use brute force method to superposescore_superpose_brute_force_with_cc = None Use cc scoring in brute force method (requires map file)ok_brute_force_cc = 0.25 OK value for brute force cc (stop looking if achieved)shift_field_distance = None Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 Å.good_enough_fraction = 0.9 Good enough fraction. If this much of model is found with sequence-based search skip ssmstarting_distance_cutoff = 2.5 Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.include_ssm = True Include SSM matching (superpose based on secondary structure)include_lsq = True Include least-squares matching (minimizing RMSD)try_lsq_first = True Try lsq matching first if lsq and ssm are both includedtry_with_fragments_if_poor_fit = True Try lsq matching with fragments if no fit obtainedinclude_swapped_match = None Try matching fixed to moving if initial

match fails
allow_reverse_connectivity = None Allow match to reversed model
direct_morph = False Directly morph moving model onto fixed model. Requires chain ID and residue numbering to be identical, but fixed_model can have missing residues. Does not allow trimming.
direct_morph_method = along_chain Choice of method for direct morph: along_chain or in_space
direct_morph_smoothing_window = 11 Smoothing window for direct morph
identify_register_shifts = False Identify positions in chains where the register between fixed and moving models changes

- superpose = True Superpose moving model on fixed model
- morph = True Morph moving model after superposition
- trim = True Trim moving model after morphing
- ssm_match_to_map = False SSM match to helices/strands from map
- superpose_brute_force = False Use brute force method to superpose
- score_superpose_brute_force_with_cc = None Use cc scoring in brute force method (requires map file)
- ok_brute_force_cc = 0.25 OK value for brute force cc (stop looking if achieved)
- shift_field_distance = None Shift field distance. Distance over which morphing leaves model mostly the same. Over longer distances morphing changes model. Default is 10 Å.
- good_enough_fraction = 0.9 Good enough fraction. If this much of model is found with sequence-based search skip ssm
- starting_distance_cutoff = 2.5 Starting distance cutoff. Try to keep atoms within this distance, but increase if too few are found.
- include_ssm = True Include SSM matching (superpose based on secondary structure)
- include_lsqr = True Include least-squares matching (minimizing RMSD)
- try_lsqr_first = True Try lsqr matching first if lsqr and ssm are both included
- try_with_fragments_if_poor_fit = True Try lsqr matching with fragments if no fit obtained
- include_swapped_match = None Try matching fixed to moving if initial match fails
- allow_reverse_connectivity = None Allow match to reversed model
- direct_morph = False Directly morph moving model onto fixed model. Requires chain ID and residue numbering to be identical, but fixed_model can have missing residues. Does not allow trimming.
- direct_morph_method = along_chain Choice of method for direct morph: along_chain or in_space
- direct_morph_smoothing_window = 11 Smoothing window for direct morph
- identify_register_shifts = False Identify positions in chains where the register between fixed and moving models changes
- alignmentSet of parameters for sequence alignment. Defaults are good for most of cases
alignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1
similarity_matrix = blosum50 dayhoff *identity
- alignment_style = local *global
- gap_opening_penalty = 1
- gap_extension_penalty = 1
- similarity_matrix = blosum50 dayhoff *identity
- controlnproc = 1 Number of processors to use. If None, uses all available

- `nproc = 1` Number of processors to use. If `None`, uses all available
- `guiGUI-specific parameter required for output directory``output_dir = None`
- `output_dir = None`

Superimposing two PDB files with superpose_pdb (deprecated, use superpose_models)

superpose_pdb.html

Author(s)

- superpose_pdb: Peter Zwart, Pavel Afonine, Ralf W. Grosse-Kunstleve

Purpose

superpose_pdb is a command line tool for superimposing one PDB model on another and writing out the superimposed model.

Usage

How superpose_pdb works:

superpose_pdb performs a least-squares superposition of two selected parts from two pdb files. If no selections are provided for fixed and moving models the whole content of both input PDB files is used for superposition. If the number of atoms in fixed and moving models is different and the models contain amino-acid residues then the sequence alignment is performed and the matching residues (CA atoms by default, can be changed by the user) are used for superposition. Note that selected (and/or matching) atoms are the atoms used to find the superposition operators while these operators are applied to the whole moving structure.

Output files from superpose_pdb

A PDB file with fitted model.

Examples

Standard run of superpose_pdb:

Running the superpose_pdb is easy. From the command-line you can type:

```
phenix.superpose_pdb fixed.pdb moving.pdb
```

Parameters can be changed from the command line:

```
phenix.superpose_pdb fixed.pdb moving.pdb selection_fixed="chain A and name CA" selection_moving="chain B and
```

Possible Problems

Specific limitations and problems:

- Different number of atoms in selection_fixed and selection_moving in case when no sequence alignment can be performed (the molecules contain no amino-acid residues) or sequence alignment failed to find matching residues.
- More than one model in one PDB file (separated with MODEL-ENDMDL)

Literature

Additional information

List of all available keywords

selection_fixed = None Selection of the target atoms to fit to (optional)selection_moving = None Selection of the atoms that will be fit to selection_fixed (optional)selection_default_moving = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")' Select protein atoms that will be used in superposition after sequence alignmentselection_default_fixed = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")' Select protein atoms that will be used in superposition after sequence alignmentreciprocal_matching = False try both orientations for homodimersinputpdb_file_name_fixed = None Name of PDB file with model to fit topdb_file_name_moving = None Name of PDB file with model that will be fit to pdb_file_name_fixedpdb_file_name_moving_other = None Name of another PDB file to transform with the same matrix as pdb_file_name_movingoutputfile_name = None Name of PDB file with model that best fits to pdb_file_name_fixedoverwrite = True Overwrite existing filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.job_title = None Job title in PHENIX GUI, not used on command linealignmentSet of parameters for sequence alignment. Defaults are good for most of casesalignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1 similarity_matrix = blosum50 dayhoff *identity

- selection_fixed = None Selection of the target atoms to fit to (optional)
- selection_moving = None Selection of the atoms that will be fit to selection_fixed (optional)
- selection_default_moving = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")' Select protein atoms that will be used in superposition after sequence alignment
- selection_default_fixed = pepnames and (name ca or name n or name c) and '(altloc " " or altloc "A")' Select protein atoms that will be used in superposition after sequence alignment
- reciprocal_matching = False try both orientations for homodimers
- inputpdb_file_name_fixed = None Name of PDB file with model to fit topdb_file_name_moving = None Name of PDB file with model that will be fit to pdb_file_name_fixedpdb_file_name_moving_other = None Name of another PDB file to transform with the same matrix as pdb_file_name_moving
- pdb_file_name_fixed = None Name of PDB file with model to fit to
- pdb_file_name_moving = None Name of PDB file with model that will be fit to pdb_file_name_fixed
- pdb_file_name_moving_other = None Name of another PDB file to transform with the same matrix as pdb_file_name_moving
- outputfile_name = None Name of PDB file with model that best fits to pdb_file_name_fixedoverwrite = True Overwrite existing filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.job_title = None Job title in PHENIX GUI, not used on command line
- file_name = None Name of PDB file with model that best fits to pdb_file_name_fixed
- overwrite = True Overwrite existing files
- target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.

- job_title = None Job title in PHENIX GUI, not used on command line
- alignmentSet of parameters for sequence alignment. Defaults are good for most of casesalignment_style = local *global gap_opening_penalty = 1 gap_extension_penalty = 1 similarity_matrix = blosum50 dayhoff *identity
- alignment_style = local *global
- gap_opening_penalty = 1
- gap_extension_penalty = 1
- similarity_matrix = blosum50 dayhoff *identity

Trimming overlapping parts of a model

trim_overlapping.html

Author(s)

- trim_overlapping: Tom Terwilliger

Purpose

Remove parts of a model that overlap with another model. Useful if you have a good model for part of the your structure and you want to add in only the new parts from another model. This can be the case if you have a good AlphaFold2 model for most of your structure and a possibly overlapping model from another source.

How trim_overlapping works:

The trim_overlapping tool identifies all the residues in your model that have a CA (or P for nucleic acids) atom close to one in the model to avoid. The cutoff distance is typically 3 Å for proteins and is adjustable. If the remaining residues lead to short segments, those segments are removed too. The shortest segment to keep is also adjustable.

Examples

Standard run of trim_overlapping:

Running trim_overlapping is easy. From the command-line you can type:

```
phenix.trim_overlapping my_model.pdb model_to_avoid.pdb
```

This will trim my_model.pdb to avoid model_to_avoid and write out my_model_trimmed.pdb

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
input_filesmodel = None Input PDB file to trim
model_selection = None If specified, only consider selected part of model
model_to_avoid = None Input PDB file to not overlap
model_to_avoid_selection = None If specified, only avoid selected part of model
model_to_avoidoutput_filetrimmed_model_prefix = None Output files with superposed models will begin with this prefix
trim_overlappingmin_dist = 5 Remove residues within this distance of target
modelminimum_fragment_length = 7 Minimum length of a segment after trimming (toss if shorter)
unique_chain_id = None Unique chain ID not present in models. Used internally
controlwrite_files = True Write output files
guiGUI-specific parameter required for output directory
output_dir = None

- `job_title` = None Job title in PHENIX GUI, not used on command line
- `input_filesmodel` = None Input PDB file to trim
`model_selection` = None If specified, only consider selected part of model
`model_to_avoid` = None Input PDB file to not overlap
`model_to_avoid_selection` = None If specified, only avoid selected part of `model_to_avoid`
- `model` = None Input PDB file to trim
- `model_selection` = None If specified, only consider selected part of model
- `model_to_avoid` = None Input PDB file to not overlap
- `model_to_avoid_selection` = None If specified, only avoid selected part of `model_to_avoid`
- `output_filetrimmed_model_prefix` = None Output files with superposed models will begin with this prefix
- `trimmed_model_prefix` = None Output files with superposed models will begin with this prefix
- `trim_overlappingmin_dist` = 5 Remove residues within this distance of target model
`minimum_fragment_length` = 7 Minimum length of a segment after trimming (toss if shorter)
`unique_chain_id` = None Unique chain ID not present in models. Used internally
- `min_dist` = 5 Remove residues within this distance of target model
- `minimum_fragment_length` = 7 Minimum length of a segment after trimming (toss if shorter)
- `unique_chain_id` = None Unique chain ID not present in models. Used internally
- `controlwrite_files` = True Write output files
- `write_files` = True Write output files
- `gui` GUI-specific parameter required for output directory
`output_dir` = None
- `output_dir` = None